

Asset Pricing With Machine Learning

Candidate

Tommaso de Martino

Supervisor

Prof. Giuseppe Ragusa



SAPIENZA
UNIVERSITÀ DI ROMA



SAPIENZA
UNIVERSITÀ DI ROMA

Asset Pricing With Machine Learning

Faculty of Economics
Department of Economics and Law
MSc in Economics and Finance

Tommaso de Martino
Student ID 1964326

Supervisor
Prof. Giuseppe Ragusa

Academic Year 2024-2025

Contents

1	Introduction	1
1.1	Equity Risk Premia	1
1.2	Objectives of the Thesis	2
1.3	What is Machine Learning?	3
1.4	Why Machine Learning for Asset Pricing?	3
1.5	Outline of Main Results	4
1.6	Literature	5
2	Data	7
2.1	Stock Characteristics	7
2.2	Macroeconomic Predictors	8
2.3	Data Standardization	8
2.4	Interactions and Feature Construction	9
3	Methodology	10
3.1	Sample Splitting	10
3.2	Objective and Loss Functions	12
3.3	Performance Evaluation	13
3.4	Variable Importance	14
4	Models	15
4.1	Linear Regression	15
4.2	Elastic Net	16
4.3	Dimension Reduction: PCR and PLS	17
4.4	Generalized Linear Models	19
4.5	Tree-Based Methods	20
4.6	Neural Networks	24
5	Results	31
5.1	Out-of-Sample R^2	31
5.2	Diebold–Mariano Tests	33
5.3	Variable Importance	35
5.4	Machine Learning Portfolios	41
5.5	Z-Score Portfolio	48
6	Conclusions	53

Appendix	55
A Details of the Characteristics	55
B Hyperparameters	59
C Algorithms	60
C.1 Lasso, Ridge, Elastic Net, and Group Lasso	60
C.2 Tree, Gradient Boosted Tree, and Random Forest	63
C.3 Neural Networks	65

1. Introduction

In this work, I conduct a comparative analysis of machine learning methods for finance, following the approach of Gu et al. (2020). The prediction of equity returns is one of the most extensively studied problems in empirical finance, owing to its importance for both asset pricing research and investment practice. At the same time, it is a particularly demanding task: the signal-to-noise ratio of financial data is notoriously low, and the number of potential predictors is very large, ranging from firm-specific characteristics to macroeconomic variables. This difficulty makes finance a particularly challenging environment for machine learning methods, where extracting stable predictive relationships is inherently difficult.

Machine learning methods provide a flexible framework to confront these challenges. Through regularization, dimension reduction, tree-based ensembles, and neural networks, they allow researchers to handle high-dimensional predictor spaces and to capture nonlinear patterns and interactions that may be difficult to detect with more restrictive specifications. The objective of this thesis is to evaluate and compare a range of such methods in a unified setting, providing insights into their relative effectiveness in forecasting stock returns.

1.1. Equity Risk Premia

Asset pricing is fundamentally concerned with determining how much an asset should be worth today given its risk and expected future returns. Within this framework, the concept of risk premia is central. Risk premia represent the portion of an asset's expected return that compensates investors for taking risk. Equivalently, they can be viewed as the excess return of a risky asset relative to a safe investment such as government bonds.

Formally, the expected return on an asset can be written as $E(r) = r_f + r_p$, where r_f is the risk-free rate and r_p is the risk premium. This decomposition highlights that expected returns depend directly on risk premia.

Therefore, understanding asset prices requires understanding expected returns, and understanding expected returns in turn requires estimating risk premia. The connection between asset pricing and risk premia is therefore immediate: asset values depend on expected returns, and expected returns are determined by the magnitude of the risk premium. In what follows, the terms expected return and equity risk premium are used interchangeably, with the latter referring to the expectation of returns in excess of the risk-free rate. Distinguishing between systematic components, idiosyncratic elements, or potential mispricing lies beyond the scope of this thesis.

1.2. Objectives of the Thesis

The purpose of this thesis is to evaluate the effectiveness of machine learning methods in forecasting equity returns, with a particular focus on their implications for measuring and exploiting equity risk premia. The analysis builds on the framework of Gu et al. (2020), and the contribution lies in assessing how different models perform when applied to return prediction and when their forecasts are used to construct investment strategies. In particular, I apply their methodological framework to a different sample period: while relying on almost the same set of predictors, the dataset used in this thesis excludes the earliest years of their sample but extends the analysis to more recent data. This allows me to investigate whether the relative performance of machine learning models is robust to changes in the time span of the data. (see Chapter 2 for a detailed discussion of the dataset).

First, I compare the predictive accuracy of a wide range of machine learning models, including penalized linear regressions, dimension reduction methods, tree-based ensembles, and neural networks. Predictive performance is measured using the out-of-sample R^2 defined in Gu et al. (2020).

Second, I evaluate the economic value of machine learning forecasts by forming portfolios based on model predictions. The analysis investigates whether strategies built on predicted returns can outperform standard benchmarks such as the excess return on the S&P 500, and whether higher out-of-sample predictive accuracy necessarily translates into higher cumulative returns.

Finally, the thesis highlights the distinction between prediction as a statistical exercise and prediction as a tool for portfolio formation. The results highlight that a model achieving a high out-of-sample predictive fit does not necessarily translate into superior portfolio returns when investment strategies are constructed from its forecasts. This underscores the need to evaluate models not only in terms of their statistical accuracy, but also with respect to their economic implications.

It is worth noting that several of the machine learning models estimated in this thesis are computationally demanding, often requiring hours to train and validate. From a practical standpoint, simpler econometric or statistical approaches can sometimes achieve comparable portfolio results at a fraction of the computational cost. For reference, a benchmark portfolio based on the cross-sectional z-score of excess returns performs similarly to some machine learning portfolios during the test period. However this outcome should not be overemphasized: the z-score portfolio captures a pattern specific to the sample, whereas machine learning methods continuously re-estimate and adapt their structure over time. Rather than fitting fixed patterns, these models learn and adapt to evolving relationships between firm characteristics and returns, providing a more flexible and realistic framework for a dynamic financial environment.

1.3. What is Machine Learning?

The term “machine learning” does not admit a single, universally agreed definition. In this thesis, I adopt a pragmatic perspective and use the term to refer to a collection of statistical methods that are primarily designed for prediction rather than inference. These methods share three common features.

First, they are well suited for high-dimensional settings, where the number of potential predictors is large and traditional econometric tools struggle with issues such as multicollinearity and overparameterization.

Second, machine learning methods typically rely on some form of regularization or variable selection to control model complexity and mitigate the risk of overfitting, thereby improving out-of-sample stability.

Third, they rely on computationally efficient algorithms that make it feasible to estimate models that would be intractable with brute-force approaches. Efficiency here is relative: even though machine learning methods are designed to scale better than traditional techniques in high-dimensional settings, training complex models on large datasets can still be computationally intensive in practice, especially when extensive hyperparameter tuning is required.

From this perspective, machine learning can be viewed as an extension of traditional econometric prediction, with greater flexibility in handling large predictor sets and richer functional forms, balanced by a careful focus on generalization performance.

1.4. Why Machine Learning for Asset Pricing?

Empirical asset pricing research can be broadly divided into two strands. The first focuses on cross-sectional asset pricing, that is, explaining why some assets are expected to earn higher returns than others. The second concerns the measurement of the aggregate equity risk premium over time. Despite their different emphasis, both perspectives share a common challenge: predicting expected returns. In practice, this means forming expectations about returns at time $t + 1$ using only information available at time t . Since prediction lies at the core of both perspectives, machine learning methods are particularly well suited to these problems, as they are designed to extract signals and improve forecast accuracy in complex, data-rich environments.

In the asset pricing literature, researchers have proposed an exceptionally large number of predictive signals. At the firm level, hundreds of stock-specific characteristics have been documented, while at the macroeconomic level, dozens of aggregate variables are considered potentially informative. As a result, the estimation of expected returns often involves datasets that are both high-dimensional and rich in candidate predictors.

A further complication is that many of these predictors are highly correlated and often

capture overlapping information. When the number of predictors grows large relative to the number of observations, traditional econometric methods may fail: they struggle to distinguish true signals from noise and are prone to overfitting. Machine learning methods are specifically designed to address such high-dimensional settings. Through variable selection techniques and dimension reduction approaches, they help control model complexity, mitigate overfitting, and extract the most relevant information from a large pool of predictors.

In a predictor-rich environment, a key challenge is the functional form through which predictors enter expected returns: should the relation be linear or nonlinear, and should we allow for interactions among predictors? The resulting space of specifications is vast, while theory offers limited guidance for selecting the most relevant features *ex ante*. Machine learning is well suited to this setting: its methodological diversity ranges from simple models to approaches that capture complex interactions; it can approximate nonlinear relationships effectively; and, through regularization, it supports conservative model selection that limits overfitting.

Following the approach of Gu et al. (2020), the empirical analysis in this thesis considers a broad set of predictive models. These include linear regression, generalized linear models with penalization, dimension reduction methods such as principal components regression (PCR) and partial least squares (PLS), as well as tree-based approaches (including regression trees, boosted trees, and random forests), and artificial neural networks.

1.5. Outline of Main Results

This section provides a brief overview of the main empirical results that will be presented in detail in Chapter 5. The analysis confirms some of the findings in Gu et al. (2020), but also highlights important differences. Neural networks and, to a lesser extent, dimension-reduction methods such as PLS and PCR, remain among the best performers in terms of predictive accuracy. However, overall out-of-sample R^2 values are lower than in the original study, suggesting that expected returns have become harder to forecast in more recent years. When model forecasts are translated into portfolio strategies, the link between predictive fit and economic performance is not always straightforward: under value weighting, GLM achieves the highest long-run returns, while under equal weighting NN4 dominates. Surprisingly, OLS does not perform as poorly in terms of portfolio returns as reported by Gu et al. (2020). Most models converge on the same groups of dominant predictors: momentum, liquidity, risk and volatility, and size. Furthermore, a simple z-score portfolio delivers competitive results despite its minimal computational cost. By contrast, machine learning models, particularly neural networks, continuously update their structure and can adapt to evolving relationships between firm characteristics and returns. Hence, even though predictive

accuracy has declined relative to Gu et al. (2020), these methods continue to add economic value by providing a flexible, data-driven framework for modeling expected returns in an increasingly efficient market environment.

1.6. Literature

The starting point of modern finance is the work of Fama (1970), who formulated the Efficient Market Hypothesis (EMH). In its strongest form, the EMH implies that excess returns should be unpredictable, since all available information is already incorporated into prices. Nevertheless, subsequent research has documented systematic return patterns that are difficult to reconcile with the strictest form of market efficiency, leading to the large literature on return predictability and anomalies (Fama and French, 2008).

This literature can be broadly divided into two strands. The first investigates differences in expected returns across stocks, i.e., the cross-sectional dimension, as documented in the extensive studies of Fama and French (2008) and Lewellen (2015). Here, the standard empirical strategy is to regress future stock returns on lagged firm characteristics.

The second strand focuses on time-series predictability of aggregate returns. Kojen and Van Nieuwerburgh (2011) survey the evidence on forecasting returns and cash flows, while Rapach and Zhou (2013) review macroeconomic predictors of stock returns. A common approach in this line of work is to run time-series regressions of portfolio returns on a small set of lagged macroeconomic variables.

Traditional econometric approaches, however, face important limitations in high-dimensional settings. When the number of potential predictors grows large and many are highly correlated, simple regressions or portfolio sorts become unreliable and prone to overfitting. This has motivated the introduction of machine learning methods into empirical asset pricing.

Early contributions applied neural networks to derivatives pricing, such as Hutchinson et al. (1994) and Yao et al. (2000), while later work extended machine learning to credit risk and consumer finance, including Khandani et al. (2010) and Butaru et al. (2016). More recently, deep learning has been used to model mortgage risk (Sadhvani et al., 2021) and portfolio selection (Heaton et al., 2016).

In the domain of stock return prediction, machine learning methods have gained increasing attention. Rapach et al. (2013) employ the lasso to predict global equity market returns using lagged returns across countries. Moritz and Zimmermann (2016) apply tree-based models for portfolio sorting, while Giglio and Xiu (2021) use dimension reduction to account for omitted factors. Shrinkage and selection methods have also been proposed to deal with large cross-sectional datasets, as in Kozak et al. (2020) and Freyberger et al. (2020).

Against this backdrop, the comparative analysis of Gu et al. (2020) represents a mile-

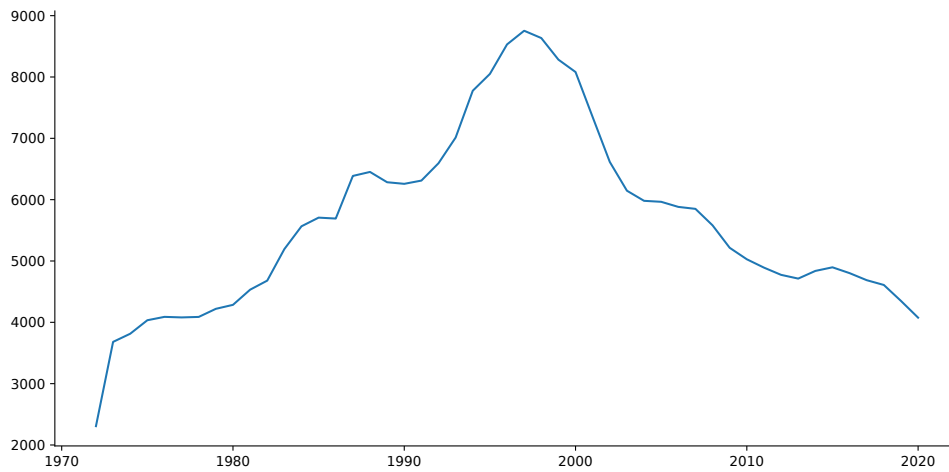
stone. They systematically evaluate a wide range of machine learning methods, including penalized regressions, dimension reduction techniques, tree-based ensembles, and neural networks, applied to both cross-sectional and time-series return prediction. Their study shows that machine learning can substantially improve predictive performance relative to traditional econometric benchmarks, with these gains translating into sizeable portfolio returns.

This thesis builds on their framework, applying the same methodological approach to a different sample period in order to examine the robustness of their findings and to explore whether the link between statistical fit and portfolio performance holds more generally.

2. Data

This chapter describes the dataset used in the empirical analysis. The sample consists of monthly stock-level observations for all common shares listed on the NYSE, AMEX, and NASDAQ exchanges, obtained from the CRSP database. The sample period extends from March 1972 to December 2020, for a total of 49 years. Over this period, the dataset includes 23,729 distinct firms, with an average of approximately 5,624 stocks per month. The one-month Treasury bill rate is used as a proxy for the risk-free rate, from which individual excess returns are computed.

Figure 1: Number of Stocks per Year



Note: The figure shows the total number of stocks available in the CRSP sample each year from 1972 to 2020. The series reflects both market entries and delistings over time.

2.1. Stock Characteristics

The construction and treatment of the data follow the framework of Gu et al. (2020). In line with their approach, I build a broad panel of stock-level predictive characteristics based on the cross-sectional return prediction literature. The dataset includes 81 firm-specific characteristics and 70 industry dummy variables based on the first two digits of the Standard Industrial Classification (SIC) code. The characteristics encompass a wide range of measures capturing firm size, valuation ratios, profitability, investment, momentum, trading frictions, and other attributes that have been shown to predict stock returns. Some of these variables are updated monthly, others quarterly or annually, depending on their underlying source and accounting frequency.

Most characteristics are released to the public with delays. To avoid forward-looking bias, monthly variables are assumed to be delayed by one month, quarterly variables

by at least four months, and annual variables by at least six months. Accordingly, when predicting returns at month $t + 1$, I use the most recent monthly characteristics available at the end of month t , the most recent quarterly data by the end of month $t - 4$, and the most recent annual data by the end of month $t - 6$.

Missing observations are handled carefully to maintain comparability across firms. For each characteristic, missing values are replaced with the cross-sectional median within the corresponding industry at each point in time. This industry-based imputation ensures that firms are compared against others operating within the same sector, thereby preserving the economic meaning of the characteristics while reducing the influence of extreme outliers.

2.2. Macroeconomic Predictors

In addition to firm-level characteristics, I include a small set of macroeconomic variables that summarize the overall state of the economy and financial markets. Following the definitions in Welch and Goyal (2008) and Gu et al. (2020), the analysis incorporates eight predictors widely used in the literature on equity premium forecasting: the dividend–price ratio (dp), earnings–price ratio (ep_m), book-to-market ratio (bmm), net equity expansion (ntis), Treasury-bill rate (tbill), term spread (tms), default spread (dfy), and stock variance (svar). These aggregate variables capture broad economic and financial conditions and complement firm-level characteristics by allowing the models to account for time variation in expected returns at the market level.

2.3. Data Standardization

Following Gu et al. (2020), I rank all firm-level characteristics cross-sectionally in each period and map these ranks to the $[-1, 1]$ interval. This normalization procedure makes the variables directly comparable across time and scales, and mitigates the impact of extreme values. By construction, the transformation preserves the ordinal information embedded in the characteristics while standardizing their dispersion. Formally, for each calendar month, all firms for a given characteristic, are ranked in ascending order based on the value of that characteristic. The lowest value receives rank 1, the highest receives rank N . Let rank_i denote the ordinal rank of firm i in that month, and N the number of firms in the cross-section. Each rank is then linearly mapped to the $[-1, 1]$ interval using the following transformation:

$$\text{scaled_rank}_i = \frac{\text{rank}_i - 1}{N - 1} \cdot 2 - 1. \quad (1)$$

This mapping ensures that the lowest rank corresponds to -1 , the highest to 1 , and all intermediate values are evenly spaced between these bounds.

2.4. Interactions and Feature Construction

The final set of predictors used in the empirical analysis combines firm-level characteristics and macroeconomic variables into a unified panel of covariates. Let $c_{i,t}$ denote the $P_c \times 1$ vector of firm-specific characteristics for stock i at time t , and let x_t denote the $P_x \times 1$ vector of macroeconomic predictors, which are common to all firms at time t . Following Gu et al. (2020), I construct the complete feature vector as the Kronecker product:

$$z_{i,t} = x_t \otimes c_{i,t}, \quad (2)$$

where $\otimes$ denotes the Kronecker product. The resulting $P = P_c P_x$ features include both the original firm-level variables and their interactions with aggregate macroeconomic conditions.

Both $c_{i,t}$ and x_t include a column of ones, which allows the feature set $z_{i,t}$ to retain pure firm-specific characteristics and pure macroeconomic variables, in addition to their interactions. The resulting representation captures how the predictive relevance of each firm-level characteristic may vary across different macroeconomic states, while preserving the interpretability of the standalone firm and macro components. After computing the Kronecker product, the constant–constant interaction (corresponding to the product of the two unit columns) is removed. Intercept terms are then handled appropriately within each individual model specification.

In total, the feature set includes $(81 + 1) \times (8 + 1) - 1 + 70 = 807$ covariates, and the dataset spans roughly 3 million firm–month observations over the 1972–2020 period.

3. Methodology

This chapter presents the methodological framework and the approach to the problem of predicting excess returns. Following the framework of Gu et al. (2020), the excess return of stock i at time $t + 1$ can be decomposed as

$$r_{i,t+1} = E_t(r_{i,t+1}) + \varepsilon_{i,t+1} \quad (3)$$

The conditional expectation of the future return can be written as

$$E_t(r_{i,t+1}) = g^*(z_{i,t}) \quad (4)$$

where $g^*(\cdot)$ denotes the optimal (but unknown) function mapping predictors $z_{i,t}$ into expected excess returns.

Stocks are indexed by $i = 1, \dots, N_t$ and months by $t = 1, \dots, T$. The aim is to estimate a function $g(z_{i,t})$ that best approximates $g^*(z_{i,t})$, maximizing the out-of-sample predictive accuracy for $r_{i,t+1}$. The predictors are denoted by the P -dimensional vector $z_{i,t}$, which contains only information available at time t .

Although this framework is highly flexible, it still relies on some key restrictions. First, the function $g^*(\cdot)$ is assumed to be invariant across both assets i and time periods t . In other words, the same functional form is applied to all stocks and over the entire sample horizon. This restriction allows the model to exploit the full cross-sectional and temporal dimension of the panel, thereby providing more stable estimates of risk premia for individual securities. This approach differs from traditional asset pricing methods, which typically re-estimate a cross-sectional model in each period or fit separate time-series models for each stock. Second, $g^*(\cdot)$ depends on the predictors only through the contemporaneous vector $z_{i,t}$. This means predictions are formed without using observations prior to period t or characteristics of stocks other than i .

3.1. Sample Splitting

Before presenting the models, it is useful to clarify how the data are partitioned and how hyperparameters are selected. These preliminary steps are crucial to guard against overfitting, which is one of the main challenges in applying machine learning methods to financial data.

Machine learning models are characterized not only by parameters to be estimated, but also by "hyperparameters" (or tuning parameters) that control model complexity. Examples include the penalization strength in lasso and elastic net, the number of boosting iterations, the number of trees in a random forest, or the depth of the trees.

There is generally little theoretical guidance on how to select hyperparameters to optimized out-of-sample performance. In line with standard practice, I determine them adaptively using information from a validation sample. To this end, the dataset is divided into three disjoint subsamples while maintaining the temporal ordering of the data.

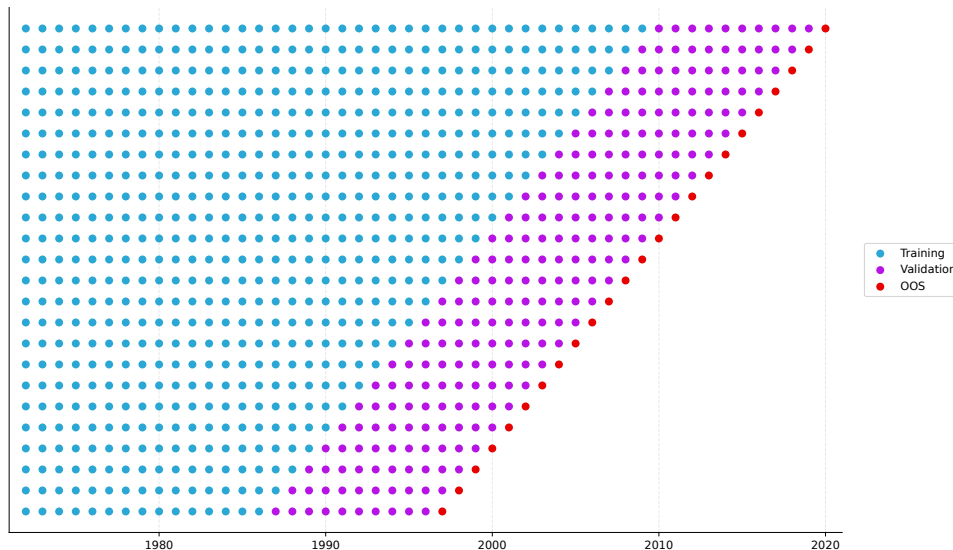
The "training set" is used to estimate model parameters given a specific choice of hyperparameters.

The "validation set" is used for hyperparameter tuning: for each candidate configuration, the model is estimated on the training data, forecasts are generated for the validation period, and the validation loss is computed. The optimal configuration is chosen as the one that minimizes the validation loss. Although the validation set provides information about predictive accuracy, it cannot be considered a true out-of-sample evaluation, since it directly affects model selection.

Therefore, a third subsample — the "test set" — is required. This portion of the data is never used for estimation or tuning and provides a genuine assessment of out-of-sample predictive performance.

Importantly, parameter estimation is always performed on the training set alone; the validation and test sets are used exclusively for model selection and performance evaluation.

Figure 2: Sample Splitting for Training, Validation, and Test Sets



Note: The figure illustrates the partitioning of the dataset into training, validation, and out-of-sample (OOS) test periods, maintaining the chronological order of observations.

The empirical analysis relies on a recursive expanding-window scheme. Each year, the training sample is augmented with newly available data and the model is refitted on the full history. The validation sample remains fixed in size while moving forward as a rolling window, and the fitted model is used to generate forecasts for the subsequent

year. This procedure preserves temporal ordering and mimics an investor's practice of updating models as information accumulates. Figure 2 provides a visual representation of this partitioning scheme, showing how the dataset is divided chronologically into training, validation, and test segments.

3.2. Objective and Loss Functions

The estimation of machine learning models requires the definition of an "objective function" to be minimized. This objective function provides the criterion that guides parameter estimation and determines how well the model approximates the true data-generating process.

All models considered in this thesis share the same baseline " l_2 " objective function, the mean squared error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T (r_{i,t+1} - g(z_{i,t}; \theta))^2 \quad (5)$$

This quadratic objective is the standard choice in the literature. Certain specifications extend the standard MSE objective by incorporating penalties on the parameter vector. The purpose of these regularization terms is to limit overfitting and to enhance the stability and accuracy of out-of-sample forecasts.

Financial data are characterized by heavy tails, which can undermine the stability of models estimated with a purely quadratic objective. To address this issue, the standard " l_2 " loss can be replaced by the Huber objective, which applies squared loss to small errors and absolute loss to large errors, thereby limiting the influence of outliers:

$$\mathcal{L}_H(\theta) = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T H(r_{i,t+1} - g(z_{i,t}; \theta), \xi), \quad (6)$$

where

$$H(x; \xi) = \begin{cases} x^2, & \text{if } |x| \leq \xi, \\ 2\xi|x| - \xi^2, & \text{if } |x| > \xi. \end{cases} \quad (7)$$

The threshold parameter, ξ , determines the switch between quadratic and linear loss, and is typically selected adaptively from the data (See Appendix B for more details.). These adjustments are designed to enhance the out-of-sample performance of machine learning methods in empirical asset pricing.

3.3. Performance Evaluation

Out-of-Sample R^2 . The primary measure of predictive accuracy is the out-of-sample R^2 , defined as

$$R_{oos}^2 = 1 - \frac{\sum_{(i,t) \in \mathcal{T}_3} (r_{i,t+1} - \hat{r}_{i,t+1})^2}{\sum_{(i,t) \in \mathcal{T}_3} r_{i,t+1}^2}, \quad (8)$$

where $\mathcal{T}_3$ denotes the test sample, which is never used for model estimation or tuning. This metric aggregates prediction errors across stocks and over time into a single panel-level measure of model fit.

This is the same definition of out-of-sample R^2 adopted by Gu et al. (2020). An important feature of this formulation is that the denominator is based on the squared excess returns without demeaning. In practice, this means that model forecasts are benchmarked against a prediction of zero, rather than against historical average returns. The rationale is that the historical mean of individual stock returns is extremely noisy and very close to zero. When excess returns are demeaned by this noisy mean, the denominator in the R^2 expression becomes artificially small, which inflates the reported predictive performance. By contrast, benchmarking against a forecast of zero provides a stricter and more meaningful standard. Indeed, Gu et al. (2020) show that when model predictions are benchmarked against historical mean returns instead of zero, the reported out-of-sample R^2 increases by roughly three percentage points.

Diebold and Mariano Test. Following Gu et al. (2020), I use the Diebold–Mariano test (Diebold and Mariano, 1995) to conduct pairwise comparisons of predictive accuracy across models. The standard Diebold and Mariano framework is designed for time-series forecasts, where dependence in forecast errors is relatively weak. In the context of stock-level return predictions, however, cross-sectional dependence can be substantial, which makes the direct application of the classical test problematic. To address this issue, I follow the approach of Gu et al. (2020) and adapt the test by comparing cross-sectional averages of prediction errors across models, rather than individual forecast errors.

Formally, to assess the relative performance of method (1) versus method (2), I define the statistic:

$$DM_{12} = \bar{d}_{12} / \hat{\sigma}_{\bar{d}_{12}}, \quad (9)$$

where

$$d_{12,t+1} = \frac{1}{n_{3,t+1}} \sum_{i=1}^{n_{3,t+1}} \left[(\hat{e}_{i,t+1}^{(1)})^2 - (\hat{e}_{i,t+1}^{(2)})^2 \right], \quad (10)$$

with $\hat{e}_{i,t+1}^{(k)}$ denoting the forecast error of model k for stock i at time $t + 1$, and $n_{3,t+1}$ the number of stocks in the testing sample at that date. The terms $\bar{d}_{12}$ and $\hat{\sigma}_{\bar{d}_{12}}$ represent, respectively, the mean and Newey–West adjusted standard error of $d_{12,t}$ over the test

period. By basing the comparison on a single time series of cross-sectionally averaged error differences, this modified version of the Diebold–Mariano test reduces serial dependence in the test statistic and is therefore more likely to satisfy the regularity conditions required for asymptotic normality, yielding valid p -values for model comparison in this panel setting. The Diebold–Mariano test is model-free and compares forecasts, not structural models.

3.4. Variable Importance

The objective of this section is to provide a modest degree of interpretability to the machine learning models. In particular, the aim is to identify which predictors exert the strongest influence on the cross-section of expected returns, conditional on the presence of all other covariates in the model.

Following Gu et al. (2020), I measure variable importance, denoted VI_j for predictor j , as the reduction in panel predictive R^2 that occurs when all values of variable j are set to zero while keeping the remaining model estimates fixed. If the exclusion of a predictor leads to a substantial decline in forecasting performance, this indicates that the variable carries important information for expected returns.

This measure is model-agnostic and can be applied consistently across the different machine learning methods considered, which makes it particularly suitable for the comparative analysis conducted in this thesis.

4. Models

This chapter describes the set of models employed in the empirical analysis. The exposition follows the structure of Gu et al. (2020): for each method, I first outline the general approach used to predict stock returns, then I present the corresponding objective function and the computational algorithm adopted for estimation. Given the wide range of algorithmic variants and technical nuances, details on the particular implementation used in the empirical analysis are deferred to Appendix C, with original sources providing further background.

4.1. Linear Regression

I start with the simplest model considered in this thesis, namely the linear regression estimated by ordinary least squares (OLS). Although this approach is not expected to perform well in a high-dimensional setting with many correlated predictors, it provides a useful benchmark. In particular, the OLS model serves as a reference point against which the benefits of more flexible and regularized machine learning methods can be evaluated. Following Gu et al. (2020), I estimate two versions of the OLS model. The first uses the full set of predictors available in the dataset, while the second is based on a restricted specification with only three pre-selected characteristics: firm size, book-to-market ratio, and momentum. The restricted model provides a useful comparison, since these three predictors are among the most widely studied factors in the asset pricing literature and are often regarded as standard benchmarks.

Model. The idea is that the function $g^*(\cdot)$ can be approximated by a linear function of the raw predictor variables and the parameter vector θ :

$$g(z_{i,t}; \theta) = z'_{i,t} \theta. \quad (11)$$

This specification corresponds to a standard linear regression and, by construction, excludes nonlinearities and interactions among predictors.

Objective Function and Computational Algorithm. The baseline estimation of this model uses the standard " l_2 " objective function given in equation (5). Minimizing this equation yields the pooled OLS estimator. However, as discussed in Section 3.2, I estimate the model using the Huber loss in equation (6), which offers greater robustness to outliers.

4.2. Elastic Net

In the context of return prediction, the number of potential predictors can be very large relative to the available sample size. As emphasized by Gu et al. (2020), in such settings the linear model may “start to overfit noise rather than extracting signal.” To guard against this problem, it is necessary to reduce the effective number of parameters. A standard approach is to augment the objective function with a penalty term. This form of regularization intentionally deteriorates in-sample fit in the hope of achieving more stable and reliable performance out-of-sample.

Model. The penalized linear model extends the standard OLS specification by augmenting the objective function with a penalty term:

$$\mathcal{L}(\theta; \cdot) = \mathcal{L}(\theta) + \phi(\theta; \cdot), \quad (12)$$

where $\mathcal{L}(\theta)$ denotes the baseline l_2 loss defined in equation (5).

Objective Function and Computational Algorithm. The literature offers different forms of penalty functions $\phi(\theta; \cdot)$. Here, I focus on the elastic net, a popular choice, specified as

$$\phi(\theta; \lambda, \rho) = \lambda(1 - \rho) \sum_{j=1}^P |\theta_j| + \frac{1}{2} \lambda \rho \sum_{j=1}^P \theta_j^2, \quad (13)$$

which depends on two non-negative hyperparameters, λ and ρ . The parameter ρ determines the balance between the two well-known regularizers. When $\rho = 0$, the penalty reduces to the lasso, which applies an l_1 norm and can shrink some coefficients exactly to zero, thereby functioning as a variable selection method. When $\rho = 1$, the penalty corresponds to ridge regression, which applies an l_2 norm that shrinks all coefficients toward zero but does not eliminate any of them. Intermediate values of ρ yield a compromise between lasso and ridge, combining selection with shrinkage.

The hyperparameters λ and ρ are tuned using the validation sample. Estimation is implemented via the accelerated proximal gradient algorithm, which can handle both least-squares and Huber loss objectives. Throughout the analysis, I adopt the Huber loss specification for its robustness to extreme observations (see Appendix C.1).

4.3. Dimension Reduction: PCR and PLS

High-dimensional predictor sets present a challenge for penalized regression methods such as lasso or elastic net, especially when predictors are highly correlated. In this case variable selection may perform poorly. For example, suppose each predictor is just the true return plus some independent noise. All variables contain the same signal, but each in a noisy form. If a method like lasso keeps only one of them, it risks relying on a predictor that looks good only by chance. Forming linear combinations of predictors, instead, preserves the signal and reduces the noise. This is the idea behind dimension reduction: combining predictors into linear components helps de-noise the information and reduce their correlation. Two classical methods for dimension reduction are principal components regression (PCR) and partial least squares (PLS).

Principal Components Regression. PCR is implemented in two stages. First, principal component analysis (PCA) transforms the original set of predictors into a reduced number of linear combinations that capture most of their covariance structure. Second, a subset of the leading components is used as regressors in a standard predictive regression. Basically, PCR regularizes estimation by discarding components associated with low variance, thereby retaining only the most informative sources of variation. A limitation of PCR is that the dimension reduction stage is not guided by the forecasting objective. The PCA transformation selects components solely on the basis of the covariance structure of the predictors, without regard to their predictive content. As a result, the components are constructed independently of how the covariates relate to future returns.

Partial Least Squares. In contrast to PCR, PLS reduces dimensionality by explicitly taking the forecast target into account. Rather than relying only on the covariance structure of the predictors, PLS constructs components that maximize their association with future returns. The asymptotic properties of PLS regression, as well as its application to forecasting risk premia, are discussed in Kelly and Pruitt (2013, 2015).

Operationally, the procedure begins by estimating, for each predictor j , its univariate regression coefficient with respect to returns. This coefficient, denoted φ_j , captures the marginal predictive power of predictor j . All predictors are then combined into a single aggregate component, with weights proportional to φ_j : strong univariate predictors receive greater weight, weak ones receive less. In this way, PLS forms components that are directly aligned with the goal of improving return forecasts. Additional components are obtained recursively. After constructing the first component, the outcome variable and all predictors are orthogonalized with respect to it. This step ensures that subsequent components are uncorrelated with those already extracted, so that each new component captures only the remaining variation not yet explained. The procedure is then repeated on the orthogonalized data until the desired number of PLS

components is obtained.

Model. The formulation of both PCR and PLS builds on the stacked representation of the linear predictive model introduced in equations (3) and (11). Specifically, the regression $r_{i,t+1} = z'_{i,t}\theta + \varepsilon_{i,t+1}$ can be expressed in compact matrix notation as

$$R = Z\theta + E, \quad (14)$$

where R denotes the $NT \times 1$ vector stacking all returns $r_{i,t+1}$, Z is the $NT \times P$ matrix of predictor vectors $z_{i,t}$, and E is the $NT \times 1$ vector collecting the residuals $\varepsilon_{i,t+1}$.

Both PCR and PLS address the dimensionality problem by projecting the original P predictors onto a smaller set of K linear combinations. In this reduced space, the forecasting model takes the form

$$R = (Z\Omega_K)\theta_K + \tilde{E}, \quad (15)$$

where Ω_K is a $P \times K$ matrix with columns $w_1, w_2, \dots, w_K$, each column w_j representing the weights used to construct the j^{th} predictive component. The transformed predictor set $Z\Omega_K$ represents a dimension-reduced version of Z , and the associated coefficient vector θ_K has dimension $K \times 1$ instead of $P \times 1$.

Objective Function and Computational Algorithm. In PCR, the weights Ω_K are obtained recursively. The j^{th} component is chosen as

$$w_j = \arg \max_w \text{Var}(Zw), \quad \text{s.t. } w'w = 1, \quad \text{Cov}(Zw, Zw_l) = 0 \text{ for } l < j, \quad (16)$$

so that each new component captures the largest possible variance in Z , subject to being uncorrelated with the components already extracted. PCR seeks K linear combinations that best reproduce the variation present in the original predictor matrix Z . The emphasis is on capturing the common structure of the predictors rather than maximizing forecasting accuracy. In practice, Ω_K can be efficiently computed via singular value decomposition (SVD) of Z , which makes the method computationally convenient.

PLS, instead, selects components that are directly targeted at prediction. The j^{th} PLS weight vector is obtained as

$$w_j = \arg \max_w \text{Cov}^2(R, Zw), \quad \text{s.t. } w'w = 1, \quad \text{Cov}(Zw, Zw_l) = 0 \text{ for } l < j. \quad (17)$$

This formulation highlights the main distinction between the two approaches: whereas PCR emphasizes variance preservation, PLS searches for directions in the predictor space that are most strongly associated with the outcome variable. In this work, the optimization problem in equation (16) is implemented using the SIMPLS algorithm of De Jong (1993).

Once the matrix of weights Ω_K is determined, both PCR and PLS estimate θ_K by re-

gressing R on the transformed predictors $Z\Omega_K$ using OLS. The number of components K is treated as a hyperparameter and selected through validation.

4.4. Generalized Linear Models

The introduction of this section begins with the model's forecast error decomposition:

$$r_{i,t+1} - \hat{r}_{i,t+1} = \underbrace{g^*(z_{i,t}) - g(z_{i,t}; \theta)}_{\text{approximation error}} + \underbrace{g(z_{i,t}; \theta) - g(z_{i,t}; \hat{\theta})}_{\text{estimation error}} + \underbrace{\varepsilon_{i,t+1}}_{\text{intrinsic error}} \quad (18)$$

where $g^*(z_{i,t})$ denotes the true but unknown forecasting function, $g(z_{i,t}; \theta)$ the functional form imposed by the econometrician, and $g(z_{i,t}; \hat{\theta})$ its estimated counterpart. The corresponding return forecast is $\hat{r}_{i,t+1} = g(z_{i,t}; \hat{\theta})$. The three components of equation (18) highlight distinct sources of forecast inaccuracy. The intrinsic error represents the unpredictable part of asset returns, reflecting randomness in financial markets, and is therefore irreducible. The estimation error arises from sampling variability; it can in principle decline with additional data, although this is not under the econometrician's control. Finally, the approximation error depends on the specification chosen by the econometrician and may be reduced by adopting more flexible functional forms that better approximate the true process. Greater flexibility, however, comes at the cost of increased risk of overfitting and potentially destabilizing out-of-sample performance.

Model. Generalized Linear Models introduce nonlinear transformations of the original predictors as new additive terms, and are therefore the closest nonlinear counterparts to the linear approaches presented in Sections 4.1 and 4.2.

I extend the simple linear specification by introducing a K -term spline expansion of the predictors:

$$g(z; \theta, p) = \sum_{j=1}^P p(z_j)' \theta_j, \quad (19)$$

where $p(\cdot) = (p_1(\cdot), p_2(\cdot), \dots, p_K(\cdot))'$ denotes the vector of basis functions, and the coefficient matrix is $\theta = (\theta_1, \theta_2, \dots, \theta_N)$ of dimension $K \times N$. Among the many possible spline constructions, I follow the approach of Gu et al. (2020), adopting a spline series of order two

$$(1, z, (z - c_1)^2, (z - c_2)^2, \dots, (z - c_{K-2})^2),$$

with knots $c_1, c_2, \dots, c_{K-2}$. This specification allows each predictor to exhibit nonlinear behavior while preserving linearity in the parameters.

Objective Function and Computational Algorithm. Since higher-order terms enter the model additively, estimation of the generalized linear specification can proceed with the same tools used for linear regression (Section 4.1). In particular, the analysis relies on a Huber loss to improve robustness against outliers. A practical challenge is that series expansions greatly increase the number of parameters to be estimated. To

control model complexity, I employ a penalization scheme specifically suited for spline expansions, known as the group lasso defined as

$$\phi(\theta; \lambda, K) = \lambda \sum_{j=1}^P \left(\sum_{k=1}^K \theta_{j,k}^2 \right)^{1/2} \quad (20)$$

The group lasso operates at the level of predictor groups: it either selects all K spline terms associated with a given characteristic, or excludes them entirely. As with the elastic net, estimation is carried out via the accelerated proximal gradient descent algorithm.

4.5. Tree-Based Methods

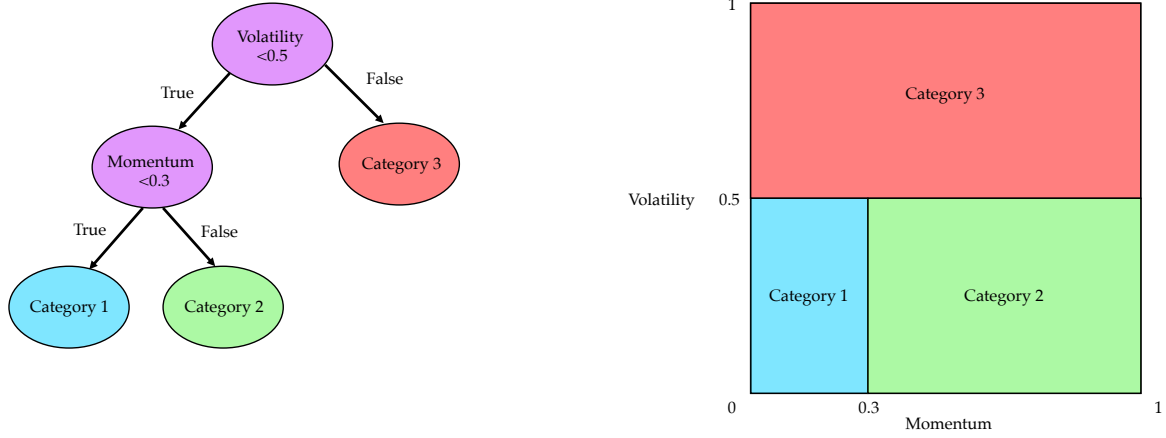
The model in (19) captures the nonlinear impact of individual predictors on expected returns but does not account for interactions among them. It is, in principle, possible to enrich the specification by introducing multivariate basis functions (i.e., basis terms that depend on more than one predictor), thereby capturing interactions. However, without a priori restrictions on which interactions to include, the parameterization grows combinatorially and estimation quickly becomes computationally infeasible.

As an alternative, regression trees provide a widely used machine learning framework for modeling multi-way interactions among predictors. In contrast to linear specifications, trees are fully nonparametric and rely on a recursive partitioning logic that differs fundamentally from standard regression approaches.

At a basic level, regression trees aim to partition the data into groups of observations with similar outcomes. The tree is built sequentially: at each step, the remaining sample is split into two bins according to a rule based on one predictor variable. Repeating this process produces a branching structure that divides the predictor space into a set of rectangular regions. Within each region, the unknown function $g^*(\cdot)$ is approximated by the average value of the outcome for the observations assigned to that region.

Figure 3, inspired by Gu et al. (2020), provides an illustrative example with two predictors, volatility and one-month momentum. The left panel shows how the tree assigns each observation to a region based on its predictor values. In the first step, the sample is split by volatility: observations with values above the 0.5 threshold are allocated to Category 3. For the remaining observations with low volatility, the next split is based on momentum. Those with momentum below 0.3 are assigned to Category 1, while those with momentum above 0.3 are placed in Category 2. The prediction for each category is then given by the average outcome of the observations contained in that partition.

Figure 3: Regression Tree



Note: The figure shows a regression tree diagram (left) and its equivalent partition of the two-predictor space (right). Terminal nodes are colored blue, green, and red, respectively. Based on these two characteristics, stocks are assigned to three categories corresponding to the colored regions.

Model. Formally, the prediction rule of a regression tree $\mathcal{T}$ with K terminal nodes (“leaves”) and maximum depth L can be expressed as

$$g(z_{i,t}; \theta, K, L) = \sum_{k=1}^K \theta_k \mathbf{1}_{\{z_{i,t} \in C_k(L)\}}. \quad (21)$$

where $C_k(L)$ denotes one of the K disjoint regions of the predictor space obtained through recursive partitioning up to depth L . Each partition corresponds to a sequence of at most L binary splitting rules applied to the predictors. The constant parameter θ_k associated with region $C_k(L)$ is defined as the sample average of the outcome variable within that region.

As an illustration, consider the example in Figure 3, with two predictors: volatility and one-month momentum. The corresponding prediction rule is

$$\begin{aligned} g(z_{i,t}; \theta, 3, 2) &= \theta_1 \mathbf{1}_{\{\text{vol}_{i,t} < 0.5\}} \mathbf{1}_{\{\text{mom1m}_{i,t} < 0.3\}} \\ &\quad + \theta_2 \mathbf{1}_{\{\text{vol}_{i,t} < 0.5\}} \mathbf{1}_{\{\text{mom1m}_{i,t} \geq 0.3\}} \\ &\quad + \theta_3 \mathbf{1}_{\{\text{vol}_{i,t} \geq 0.5\}}. \end{aligned} \quad (22)$$

Objective Function and Computational Algorithm. To grow a tree means deciding how to partition the dataset into subregions (often called “bins” or “partitions”). Each split aims to separate observations with different outcomes so that, within each partition, the outcomes are as homogeneous as possible. At each step, the algorithm must determine both the predictor variable to split on (e.g., volatility, momentum, size) and the threshold value (e.g., $\text{vol} < 0.5$ or $\text{mom} \geq 0.3$). This decision is made in a greedy

way: the procedure selects the split that provides the largest immediate reduction in prediction error, evaluated locally at the current stage of the tree. The vast number of possible tree configurations makes exact optimization infeasible. Instead, the literature relies on heuristic algorithms designed to efficiently approximate the optimal tree structure.

In this work, I rely on the algorithm of Breiman et al. (1984), a standard reference for tree-based methods. Further implementation details are provided in Appendix C.2. The tree grows by recursively splitting the data into subsets that improve predictive accuracy. At each node, the algorithm selects a predictor and a threshold value in a greedy fashion, focusing only on the split that gives the largest immediate gain. The quality of a split is assessed through an “impurity” measure, which evaluates how homogeneous the outcomes are within a node. In this work I use the standard quadratic impurity, defined for a subset C as

$$H(\theta, C) = \frac{1}{|C|} \sum_{z_{i,t} \in C} (r_{i,t+1} - \theta)^2, \quad (23)$$

where $|C|$ is the number of observations in C . The optimal prediction for the node is simply the sample mean,

$$\theta^* = \frac{1}{|C|} \sum_{z_{i,t} \in C} r_{i,t+1}.$$

Hence, the algorithm searches for the split that minimizes the impurity across the resulting partitions. This recursive partitioning continues until a stopping rule is reached, such as a maximum depth, a limit on the number of leaves, or a minimum node size. These thresholds are chosen adaptively using a validation sample, in order to balance model flexibility with the risk of overfitting. Tree-based models offer several advantages. They are unaffected by monotonic transformations of the predictors, can handle both categorical and continuous variables within the same framework, and are capable of capturing highly nonlinear relationships. Moreover, a tree of depth L can represent interactions among up to $(L - 1)$ predictors. At the same time, this flexibility makes them highly susceptible to overfitting, which necessitates strong regularization. To address this issue, the empirical analysis follows Gu et al. (2020) and employs two ensemble methods that mitigate overfitting by aggregating predictions from multiple trees into a single forecast.

Boosting. The first regularization method is boosting, originally introduced by Schapire (1990) and Freund (1995) in the context of classification problems to enhance the performance of weak learners. Friedman et al. (2000) and Friedman (2001) extended the approach beyond classification, leading to the development of gradient boosted regression trees (GBRT). Boosting works by recursively combining the forecasts of many

very simple trees. Individually, such shallow trees are weak learners with very limited predictive power. The key idea is that, when aggregated, a large number of weak learners can form a single strong predictor that is more stable and accurate than a complex tree fitted in one step.

In the gradient boosted regression tree (GBRT) framework, the procedure begins by fitting a shallow tree to the training sample. This initial tree provides a rough forecast with high bias. A second tree, of the same shallow depth L , is then used to fit the residuals left unexplained by the first tree. The prediction from this second tree is added to the ensemble, but scaled by a shrinkage factor $\nu \in (0, 1)$ to reduce the risk of overfitting. The process is repeated: at each iteration b , a new shallow tree is fitted to the residuals from the model with $b - 1$ trees, and its contribution is added to the total forecast, again weighted by ν . After B iterations, the final model is an additive combination of B shallow trees, governed by three tuning parameters: the tree depth L , the learning rate ν , and the number of boosting rounds B . These parameters are selected adaptively using the validation sample.

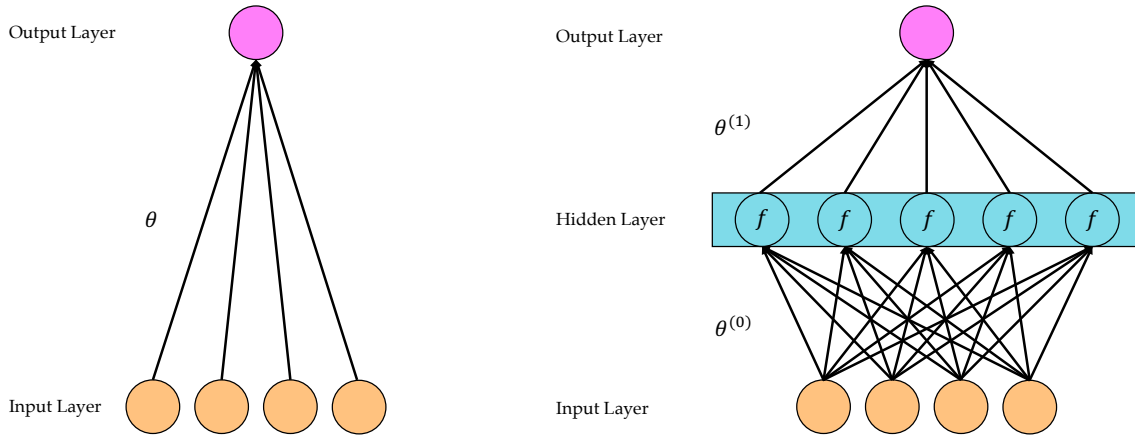
Random Forest. Random forests are an ensemble method that combines forecasts from many decision trees. They can be viewed as a refinement of the general bootstrap aggregation (bagging) procedure introduced by Breiman (2001). In the bagging framework, B bootstrap samples are drawn from the data, and a separate regression tree is estimated for each sample. The final prediction is obtained by averaging across the forecasts of all trees. Individual trees fitted on bootstrap samples tend to be deep and prone to overfitting, which makes their predictions highly variable. By averaging across many such forecasts, bagging substantially reduces this variance and stabilizes predictive performance. Random forests extend the bagging procedure by adding a mechanism that reduces correlation among trees. In plain bagging, if a single variable (for example, volatility) is particularly strong, many trees will tend to split on it at an early stage, leading to highly correlated predictions and limiting the benefits of averaging. Random forests mitigate this issue by restricting the set of candidate predictors considered at each split: only a random subset of variables is eligible at any branch. As a result, some trees are forced to split on different characteristics, which reduces the average correlation among trees and enhances the variance reduction compared to standard bagging.

The depth of the trees L and the number B of bootstrap are the tuning parameters selected in the validation sample. Details of the implementation are described in Appendix C.2.

4.6. Neural Networks

The final nonlinear method considered is the artificial neural network (ANN). Neural networks are among the most powerful tools in machine learning, supported by strong theoretical results that establish their ability to serve as universal approximators for smooth functions (Hornik et al., 1989; Cybenko, 1989). Their strength lies in the flexibility to capture complex nonlinear relationships through layered transformations of the predictors. At the same time, this flexibility entails substantial drawbacks: neural networks are often highly parameterized, difficult to interpret, and generally rank among the least transparent modeling approaches.

Figure 4: Neural Networks



Note: The figure illustrates two simple neural network architectures, one without (left) and one with (right) a hidden layer. Orange circles represent the input layer, while light purple circles represent the output layer. Arrows correspond to weight parameters connecting the nodes. In the network including a hidden layer, the inputs are first transformed through a nonlinear activation function f before being passed to the output.

Model. In this work I focus on standard feed-forward neural networks. These models are organized into an input layer of raw predictors, one or more hidden layers that combine and nonlinearly transform the predictors, and an output layer that maps the hidden representations into the final prediction. Figure 4 provides two illustrative examples.

The input layer contains as many units as the number of predictors; in the illustrative example this corresponds to four inputs, $z_1, \dots, z_4$. The left panel depicts the simplest possible network, which does not include hidden layers. In this case, each predictor is directly linked to the output through a parameter vector θ of dimension five, consisting of an intercept term and one weight assigned to each predictor. The output layer combines the weighted inputs into the prediction $\theta_0 + \sum_{k=1}^4 z_k \theta_k$, which shows that the most basic neural network architecture is equivalent to a linear regres-

sion model.

Greater flexibility can be introduced by adding hidden layers between the input and output. As illustrated in the right panel of Figure 4, the network includes one hidden layer with five neurons. Each neuron receives a linear combination of all input variables, as in the simple architecture on the left, and then applies a nonlinear activation function $f(\cdot)$ to the aggregated signal before transmitting it to the next layer. Each neuron in the hidden layer applies a nonlinear transformation to a weighted combination of the input variables.

Consider the second neuron in the hidden layer. Its output is obtained by first computing a weighted sum of all the input variables, adding a bias term, and then applying the nonlinear activation function $f(\cdot)$:

$$x_2^{(1)} = f \left(\theta_{2,0}^{(0)} + \sum_{j=1}^4 z_j \theta_{2,j}^{(0)} \right).$$

Here, $\theta_{2,0}^{(0)}$ denotes the bias term, $\theta_{2,j}^{(0)}$ are the weights linking the inputs z_j to the second hidden unit, and $f(\cdot)$ represents the nonlinear activation. Once the hidden layer has produced its activations $x_1^{(1)}, \dots, x_5^{(1)}$, the network aggregates them linearly to generate the forecast:

$$g(z; \theta) = \theta_0^{(1)} + \sum_{j=1}^5 x_j^{(1)} \theta_j^{(1)},$$

where $\theta_0^{(1)}$ is the bias at the output layer and $\theta_j^{(1)}$ are the weights connecting the hidden layer to the output. This setup illustrates how neural networks generalize linear regression by inserting intermediate nonlinear transformations between the inputs and the final prediction. In the network of Figure 4, there are 4 input variables and one hidden layer with 5 neurons. Each hidden neuron receives a weighted signal from all inputs plus a bias term. Hence, the first (input→hidden) layer has

$$5 \times (4 + 1) = 25$$

parameters (four input weights + one bias for each of the five neurons). The output layer linearly aggregates the 5 hidden activations and includes its own bias, for

$$5 + 1 = 6$$

additional parameters. Summing up,

$$25 + 6 = 31$$

parameters in total.

Designing a neural network involves several modeling choices, such as the number of hidden layers, the number of neurons per layer, and the connectivity pattern among units. Although the universal approximation theorem guarantees that a single hidden layer is theoretically sufficient to approximate any smooth function (Hornik et al., 1989; Cybenko, 1989), recent work has emphasized the benefits of depth. In particular, Eldan and Shamir (2016) show that even a modest increase in depth can be exponentially more effective than simply widening a shallow network. Following the breakthrough contribution of Hinton et al. (2006), research in machine learning has increasingly adopted deeper architectures, with state-of-the-art models employing over a hundred layers for tasks such as image recognition (He et al., 2016). By contrast, when the available dataset is relatively small, simpler network architectures with only a few hidden layers and limited numbers of neurons often yield superior performance. Very deep networks can be difficult to train effectively: the large number of parameters increases the risk of overfitting, the loss function is highly non-convex, and the recursive computation of derivatives, called “back-propagation”, may suffer from numerical instabilities such as exploding or vanishing gradients.

Since tuning all parameters through the validation set would be computationally prohibitive, I follow the approach of Gu et al. (2020) and fix a set of network architectures *ex ante*, estimating each of them. According to the setup of Gu et al. (2020), I consider neural networks with up to five hidden layers. The shallowest specification, denoted NN1, consists of a single hidden layer with 32 neurons. NN2 includes two hidden layers with 32 and 16 neurons, respectively. NN3 extends this structure to three hidden layers with 32, 16, and 8 neurons. NN4 further adds a fourth layer, resulting in 32, 16, 8, and 4 neurons. Finally, NN5 is the deepest network considered, comprising five hidden layers with 32, 16, 8, 4, and 2 neurons, respectively. The number of neurons in each layer is chosen according to the geometric pyramid rule (Masters, 1993). All architectures are fully connected, meaning that each unit receives input from all units in the layer below.

Several alternatives exist for the choice of nonlinear activation function, including sigmoid, hyperbolic tangent, and softmax. In line with much of the recent literature, I adopt the rectified linear unit (ReLU) as the activation function for all hidden units, defined as

$$\text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0, \\ x & \text{if } x \geq 0, \end{cases}$$

ReLU has become widely used because it tends to promote sparsity in the set of active neurons and permits efficient computation of derivatives, which accelerates training.

The structure of the neural network can be formalized as follows. Let $K^{(l)}$ denote the number of neurons in layer $l = 1, \dots, L$, and let $x_k^{(l)}$ be the output of the k -th neuron

in that layer. For convenience, define the vector of activations in layer l , augmented with a constant term, as

$$x^{(l)} = (1, x_1^{(l)}, \dots, x_{K^{(l)}}^{(l)})'. \quad (24)$$

The input layer is initialized analogously using the raw predictors:

$$x^{(0)} = (1, z_1, \dots, z_P)'. \quad (25)$$

For each layer $l > 0$, the output of neuron k is obtained recursively by applying the activation function to the weighted signal:

$$x_k^{(l)} = \text{ReLU}(x^{(l-1)'} \theta_k^{(l-1)}), \quad (26)$$

and the final prediction of the network is given by

$$g(z; \theta) = x^{(L-1)'} \theta^{(L-1)}. \quad (27)$$

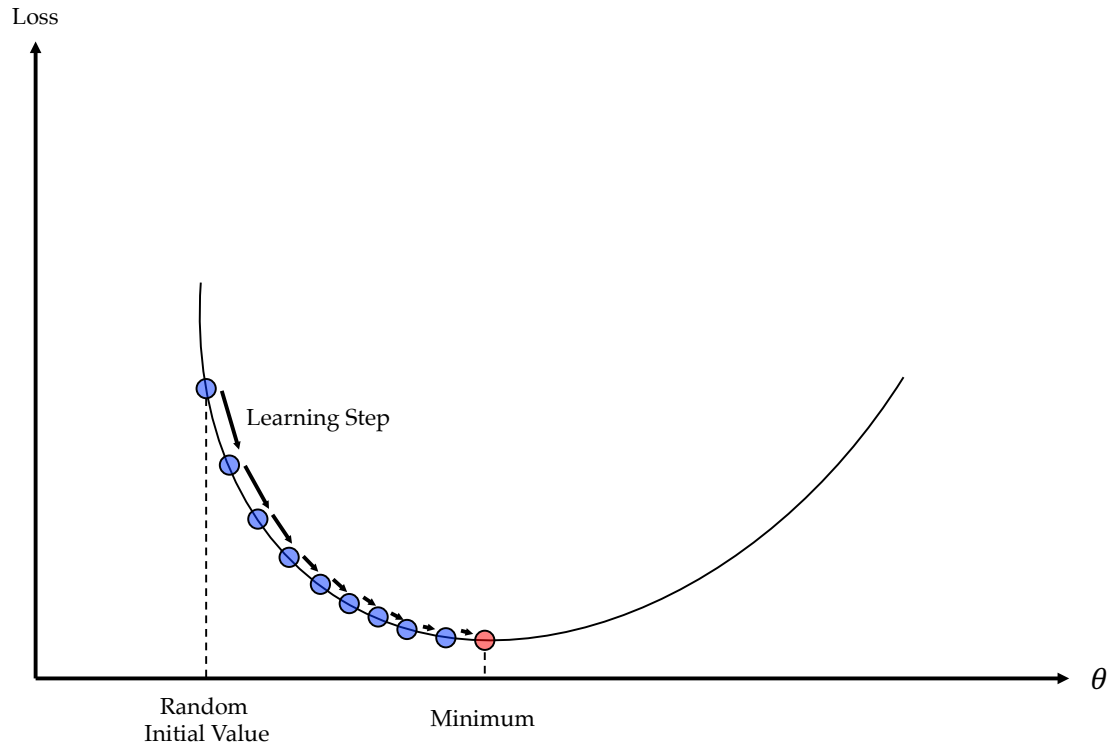
The number of weight parameters in layer l is $K^{(l)}(1 + K^{(l-1)})$, reflecting one bias and $K^{(l-1)}$ weights per neuron. The output layer adds a further $1 + K^{(L-1)}$ weights.

Objective Function and Computational Algorithm. The weights of the neural network are estimated by minimizing a penalized l_2 objective function of prediction errors. In contrast to tree-based methods, where optimization is performed sequentially through a series of greedy local splits, neural networks allow for joint updates of all parameters at once through gradient-based optimization. This property makes training a neural network, at least in principle, more flexible and potentially more efficient, as the parameter estimates are adjusted in a global rather than purely local fashion. However, the strong nonlinearity and nonconvex nature of neural networks, combined with their large number of parameters, render exact optimization computationally demanding, often prohibitively so. A widely adopted approach is to train the network using stochastic gradient descent (SGD). Rather than computing the gradient of the loss function on the entire training set at each iteration, SGD relies on randomly selected subsets of the data to approximate the gradient. This approach achieves a substantial reduction in computational cost at the expense of introducing some noise in the gradient estimate.

Figure 5 illustrates the intuition behind the gradient descent algorithm. At its core, the method relies on the gradient. In one dimension, the gradient reduces to the slope of the function, i.e., its derivative. In higher dimensions, it is the natural extension of the derivative: a vector pointing in the direction where the loss increases most rapidly at the current parameter values. Gradient descent updates the parameters in the opposite direction of the gradient, since this is the direction along which the loss decreases most rapidly. At each iteration, the gradient of the loss with respect to the current pa-

rameters is evaluated and the parameters are adjusted by a small step, the learning rate. Repeating these updates drives the parameters progressively toward a (typically local) minimum of the loss.

Figure 5: Gradient Descent

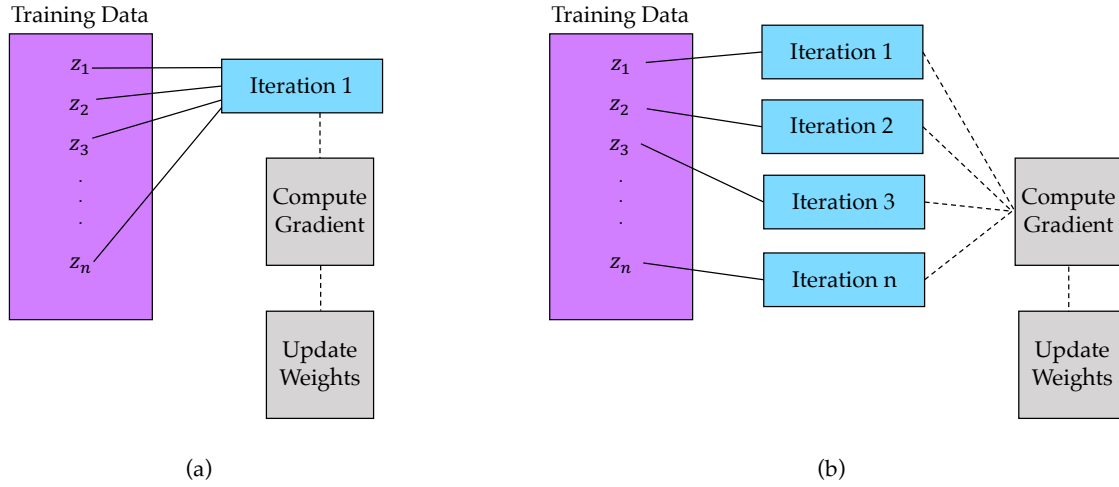


Note: The figure provides a schematic illustration of gradient descent. Starting from a random initialization of the parameters, the algorithm takes iterative steps that reduce the loss function until reaching a minimum.

Figure 6 illustrates the difference between standard gradient descent and stochastic gradient descent. Panel (a) depicts batch gradient descent, where at each iteration the gradient of the loss function is computed using the entire training dataset. While this guarantees that every update is based on the true direction of steepest descent, it also makes the procedure computationally demanding when the dataset is large.

Panel (b) shows stochastic gradient descent (SGD). Instead of processing the full dataset, each update is based on the gradient computed from a single randomly chosen observation. This introduces noise into the update direction, so the trajectory toward the minimum is more irregular. However, the randomness can help the algorithm escape shallow local minima, and the much lower computational cost per iteration makes SGD highly efficient in practice for large-scale problems.

Figure 6: Gradient Descent vs. Stochastic Gradient Descent



Note: Panel (a) illustrates batch gradient descent, where gradients are computed on the entire training set at each step. Panel (b) shows stochastic gradient descent, where updates rely on individual observations, leading to noisier but computationally cheaper steps.

In practice, rather than relying exclusively on single observations, it is common to compute gradients using small random subsets of the data, known as batches. This approach, called batch SGD, strikes a balance between full-batch gradient descent and purely stochastic updates. By computing the gradient on a subset of observations, batch methods reduce the randomness of standard SGD while remaining far less computationally demanding than full-batch training. The mini-batch size itself is a hyperparameter: smaller batches yield noisier but faster updates, whereas larger batches provide smoother but more expensive updates. In modern applications, mini-batch SGD is generally preferred as it offers a robust compromise between stability and efficiency.

Due to the strong nonlinearity and high degree of parameterization, neural networks are particularly prone to overfitting and therefore require careful regularization. Following the approach of Gu et al. (2020), in addition to standard ℓ_1 penalization of the weights, the estimation strategy combines several complementary techniques: learning rate shrinkage, early stopping, batch normalization, and the use of ensembles.

A central hyperparameter in stochastic gradient descent is the learning rate, which determines the step size of parameter updates. If the learning rate remains fixed while the gradient becomes small, random noise in its estimation can dominate the signal, leading to unstable training. To address this issue, I follow the adaptive scheme of Kingma (2014), which automatically shrinks the learning rate as optimization progresses (see Appendix C.3).

Another widely used regularization device is early stopping. The procedure begins with an initial guess of the parameters, typically set close to zero to enforce parsimony. During training, the parameters are gradually updated in order to reduce the predic-

tion error on the training sample. At the same time, forecasts are generated for the validation sample. Once the validation error stops decreasing and begins to rise, the optimization is terminated. The idea is that continuing to train beyond this point would only improve the in-sample fit at the cost of worse out-of-sample performance. By interrupting the optimization before full convergence, early stopping shrinks the estimates toward their initial values. It can be used as a stand-alone form of regularization or combined with penalization methods, as in Gu et al. (2020).

Batch normalization (Ioffe and Szegedy, 2015) is designed to address the problem of internal covariate shift, where the distribution of inputs in the hidden layers changes as parameters are updated. In practice, this means that the inputs received by hidden units during training may follow distributions that differ substantially from those observed in the validation sample, especially in deep networks with many parameters and complex structures. At each training step, the algorithm cross-sectionally de-means and variance-standardizes the inputs to every hidden unit within the current batch, thereby stabilizing their scale and preserving the representational capacity of the network.

Finally, following the ensemble learning literature (Hansen and Salamon, 2002; Dietterich, 2000), I adopt an ensemble approach in training neural networks. Specifically, I estimate multiple networks initialized with different random seeds and obtain the final prediction by averaging the forecasts across all of them. This strategy mitigates the variance of individual models, as the stochastic nature of the optimization procedure can lead different initializations to converge to distinct solutions.

5. Results

This chapter presents the empirical findings of the study, evaluating both the statistical and economic performance of the models introduced in the previous chapters.

First, I assess the out-of-sample predictive accuracy of each model using the R^2_{oos} metric and formal pairwise Diebold–Mariano tests to determine whether differences in forecast performance are statistically significant. Second, I examine variable-importance patterns to identify which characteristics contribute most to return predictability. Third, I evaluate the economic implications of the forecasts by constructing value-weighted and equally weighted portfolios based on predicted returns, comparing the realized performance across models. Finally, I introduce a simple, model-free benchmark based on cross-sectional z-scores of excess returns, designed to test whether complex machine learning models are truly necessary to achieve strong portfolio results.

These analyses provide a comprehensive view of how statistical predictability translates into economic value and clarify the relative advantages of different modeling approaches.

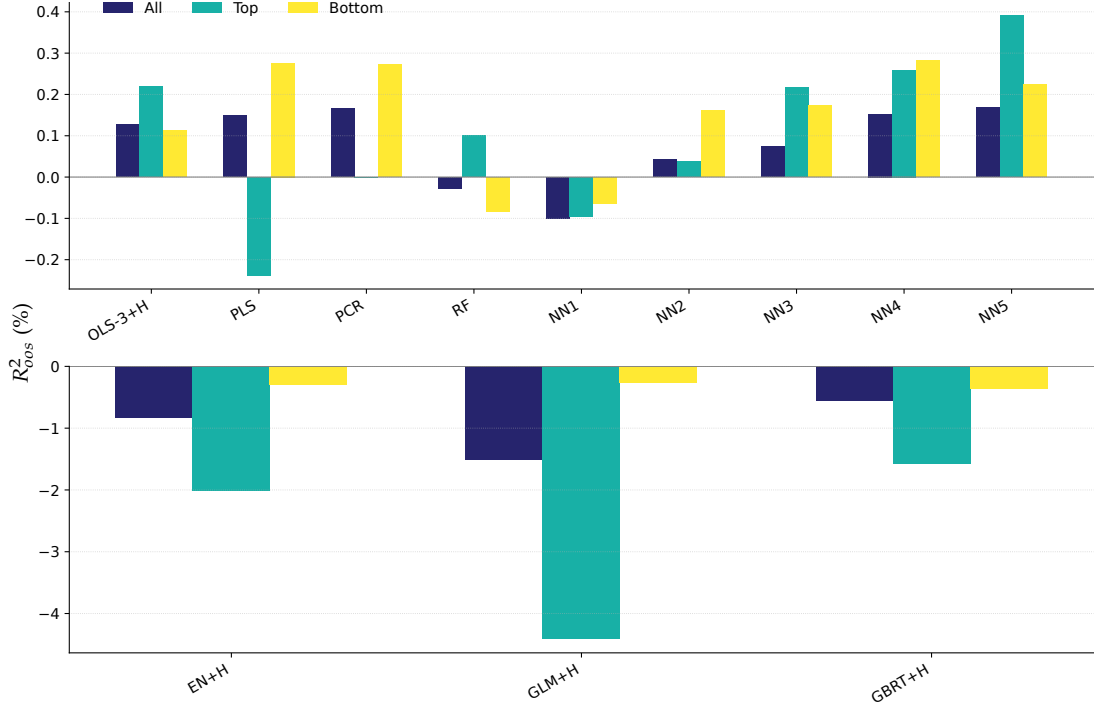
5.1. Out-of-Sample R^2

Table 1 reports the out-of-sample predictive performance of all models in terms of percentage R^2_{oos} . The comparison includes thirteen specifications: OLS with the full set of predictors, a restricted OLS model using only size, book-to-market ratio, and momentum (OLS-3), partial least squares (PLS), principal components regression (PCR), elastic net (EN), generalized linear model with group lasso (GLM), random forest (RF), gradient boosted regression trees (GBRT), and five neural network architectures of increasing depth (NN1–NN5). For OLS, EN, GLM, and GBRT, results refer to the robust versions estimated with the Huber loss, which yield superior performance relative to the standard quadratic specification. The first row shows results for the full cross-section of stocks, while the subsequent rows report R^2_{oos} values for the top and bottom 1,000 firms by market capitalization.

The results in Table 1 reveal substantial variation in predictive accuracy across models. The OLS model estimated on the full set of predictors performs poorly, yielding a strongly negative out-of-sample R^2_{oos} of -4.35% , which indicates that an unregularized specification fails to generalize beyond the training sample. By contrast, the restricted OLS model (OLS-3), which relies only on size, book-to-market ratio, and momentum, performs markedly better with an R^2_{oos} of 0.13% . This improvement highlights the benefits of parsimony when dealing with high-dimensional predictor spaces.

Table 1: Monthly Out-of-sample Stock-level Prediction Performance (Percentage R^2_{oos})

	OLS	OLS-3	PLS	PCR	EN	GLM	RF	GBRT	NN1	NN2	NN3	NN4	NN5
	+H	+H			+H	+H		+H					
All	-4.35	0.13	0.15	0.17	-0.83	-1.51	-0.03	-0.56	-0.10	0.04	0.07	0.15	0.17
Top 1000	-11.83	0.22	-0.24	0.00	-2.01	-4.42	0.10	-1.58	-0.10	0.04	0.22	0.26	0.39
Bottom 1000	-2.11	0.11	0.28	0.27	-0.30	-0.27	-0.08	-0.36	-0.07	0.16	0.17	0.28	0.22



Note: Monthly out-of-sample R^2 for the entire panel of stocks. “+H” indicates the use of Huber loss instead of the standard l_2 loss. R^2 values are also reported for subsamples including only the top 1,000 and bottom 1,000 stocks by market value. The lower panel provides a visual comparison of the R^2 statistics shown in the table (OLS omitted due to its large negative values). For clarity of presentation, the figure is split into upper and lower panels, as the range of R^2 values differs substantially across models.

Among the linear models, PLS and PCR achieve slightly higher predictive performance (0.15% and 0.17%, respectively), suggesting that dimension reduction helps stabilize estimation without introducing excessive bias.

The penalized extensions deliver mixed results. Elastic net and the generalized linear model both exhibit negative R^2_{oos} values, indicating that regularization alone does not substantially improve predictive accuracy in this setting.

Tree-based methods display contrasting results. The random forest achieves R^2_{oos} values that are slightly negative but close to zero. In contrast, the gradient boosted regression trees (GBRT) perform consistently worse, producing negative R^2_{oos} across all subsamples. Overall, both approaches fail to outperform a naive benchmark that predicts zero excess returns.

Neural networks show a clearer pattern. Increasing network depth is associated

with improved out-of-sample performance. The shallower architectures (NN1–NN3) perform slightly worse than the restricted OLS model with only three characteristics, while the deeper networks (NN4 and NN5) match the predictive accuracy of the best-performing linear models, namely PLS and PCR.

When comparing performance across size-sorted subsamples (Top 1,000 vs. Bottom 1,000 stocks), no clear or systematic pattern emerges. Some models perform slightly better among larger firms, while others achieve higher R^2_{os} for smaller firms. This indicates that model performance is largely independent of firm size, with no consistent advantage in predicting either large- or small-cap stocks.

The comparable performance of PLS and PCR relative to more complex models likely reflects the noisy nature of stock returns and the high degree of multicollinearity among predictors. With limited true signal and many correlated variables, these dimension-reduction methods effectively extract the common components of variation while avoiding overfitting, which allows them to generalize as well as more flexible approaches such as neural networks.

These findings contrast with those of Gu et al. (2020), who report markedly higher predictive performance for tree-based methods and neural networks relative to PLS and PCR, suggesting that model rankings may be sensitive to the sample period or data construction.

5.2. Diebold–Mariano Tests

To formally evaluate whether differences in predictive accuracy across models are statistically significant, Table 2 reports pairwise Diebold–Mariano (DM) test statistics for all model combinations. Positive values indicate that the model listed in the column achieves a lower mean squared prediction error than the model in the corresponding row, implying superior out-of-sample performance. The tests are conducted in a one-sided form, where the alternative hypothesis states that the column model outperforms the row model. Under the null hypothesis of equal predictive accuracy, the statistic follows approximately a standard normal distribution, $\mathcal{N}(0, 1)$. The test statistic is computed using Newey–West standard errors with six lags to account for potential serial correlation in forecast errors (results are virtually unchanged when using twelve lags).

Values greater than 1.65 indicate significance at the conventional 5% one-sided level (bold entries). To account for multiple comparisons, I apply a conservative Bonferroni adjustment. With thirteen models in total, each model is compared against twelve others, yielding twelve effectively independent pairwise tests. Following Gu et al. (2020), I therefore divide the nominal 5% significance level by twelve ($0.05/12 \approx 0.0042$), which corresponds to a Bonferroni-adjusted one-sided critical value of approximately 2.64. Statistics exceeding this threshold are marked with an asterisk. Negative values indicate that the row model attains lower prediction errors than the column model, but

Table 2: Comparison of Monthly Out-of-Sample Prediction using Diebold–Mariano Tests

	OLS-3 +H	PLS	PCR	EN +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
OLS+H	2.72*	2.78*	2.80*	1.90	1.29	2.60	2.23	2.76*	2.77*	2.81*	2.83*	2.79*
OLS-3+H		0.13	0.55	-1.25	-1.10	-1.30	-4.23	-1.79	-1.32	-0.63	0.24	1.07
PLS			1.33	-1.18	-1.07	-1.26	-3.46	-2.26	-2.05	-0.99	0.24	0.90
PCR				-1.22	-1.09	-1.36	-3.55	-2.60	-2.63	-1.46	-0.27	0.39
EN+H					-0.73	0.87	0.11	0.75	0.97	1.04	1.18	1.28
GLM+H						0.92	0.57	0.84	0.96	0.99	1.06	1.11
RF							-2.78	-0.24	0.51	0.64	1.18	1.47
GBRT+H								1.65	2.62	2.59	3.22*	3.89*
NN1									1.66	3.81*	3.26*	2.67*
NN2										0.81	3.19*	2.39
NN3											1.66	1.42
NN4												0.65

Note: This table reports pairwise Diebold–Mariano test statistics comparing the out-of-sample stock-level prediction performance among thirteen models. Positive numbers indicate the column model outperforms the row model. Bold font indicates the difference is significant at the 5% level or better for individual tests. An asterisk indicates significance at the 5% level for 12-way comparisons via the conservative Bonferroni adjustment.

they are not highlighted since the test is directional. Only the upper triangular part of the symmetric test matrix is reported so that each comparison appears once.

Consistent with the results from the R^2_{os} analysis, the tests confirm that most machine learning models significantly outperform the unregularized OLS benchmark, reinforcing the conclusion that naïve linear estimation does not generalize well out of sample. Among the more flexible methods, differences in predictive accuracy are generally modest and rarely statistically significant, suggesting that no single model consistently dominates across the entire panel.

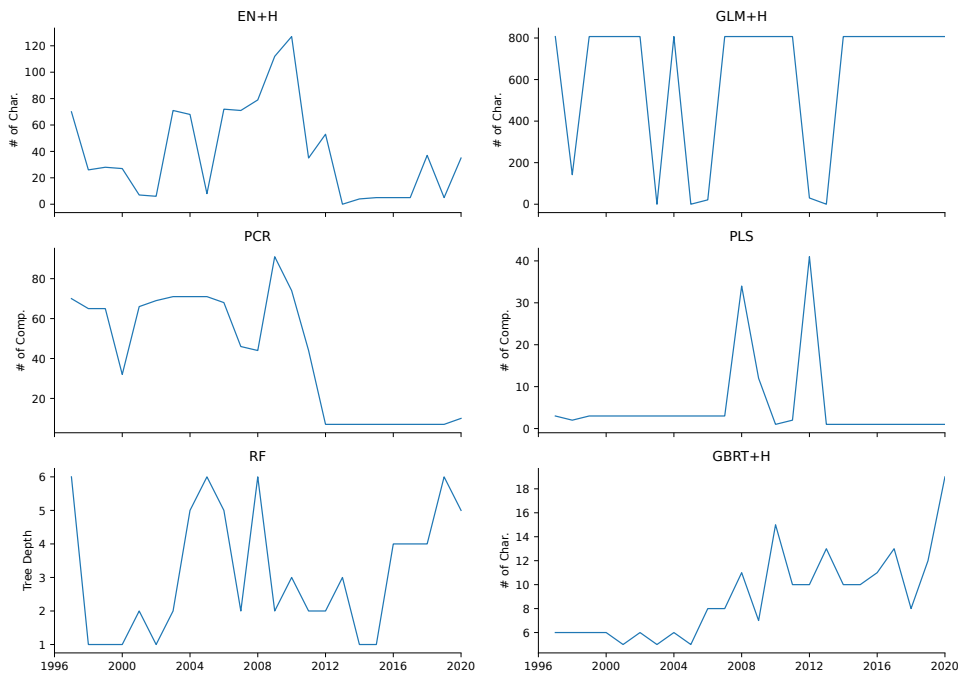
Random forest and gradient boosted regression trees (GBRT) exhibit weak relative performance. GBRT outperforms only the unregularized OLS benchmark, and even that difference is not significant once the Bonferroni correction is applied. Random forest displays a similar pattern, with no meaningful improvement over other machine learning models.

By contrast, all neural network architectures outperform GBRT, and the deeper networks (NN4 and NN5) remain significantly superior even under the conservative Bonferroni adjustment. None of the neural networks, however, significantly outperform the dimension–reduction methods (PLS and PCR), although deeper architectures (NN4 and NN5) consistently outperform the shallower ones (NN1 and NN2), suggesting that additional layers contribute to better generalization without overfitting.

5.3. Variable Importance

Before examining which predictors contribute most to the performance of each model, it is useful to first assess the effective complexity of the models over time. Figure 7 illustrates how model complexity evolved at each re-estimation date throughout the out-of-sample period.

Figure 7: Evolution of Model Complexity Over Time



Note: The figure illustrates model complexity across the 24-year out-of-sample evaluation period. Complexity is measured differently according to the structure of each model. Specifically, EN and GLM report the number of features with non-zero coefficients, PCR and PLS indicate the number of selected components, RF shows the average tree depth, and GBRT reflects the number of distinct characteristics entering into the trees.

Across models, a common pattern emerges around the 2008 financial crisis, when model complexity temporarily increases before declining in subsequent years. This behavior likely reflects the greater instability of the return-generating process during that period, which led models to rely on a larger number of features or components. Beyond this episode, however, no persistent patterns are evident.

A notable exception is the generalized linear model (GLM), which displays an irregular behavior—alternating between periods with nearly all predictors active and phases with very few selected variables—suggesting instability in its tuning dynamics.

In contrast, the elastic net (EN), partial least squares (PLS), and principal components regression (PCR) tend to favor much simpler specifications after 2008, consistently selecting a limited number of features or components.

This provides context for interpreting variable importance, as models with higher flexibility may rely on a broader and more dynamic set of predictors.

Figure 8 displays the estimated variable importance for the top 20 most influential firm-level characteristics across models. Variable importance is computed as described in Section 3.4, based on the reduction in out-of-sample R^2 resulting from setting all values of a given predictor to zero within each training sample. These reductions are then averaged over time to obtain a single importance measure for each predictor. Within each model, importances are normalized to sum to one, providing a measure of relative importance that facilitates comparison across predictors.

The variable-importance results in Figure 8 reveal that a relatively small number of firm-level characteristics drive most of the predictive power across models. These characteristics can be grouped into four main categories: momentum, liquidity, risk and volatility, and firm size.

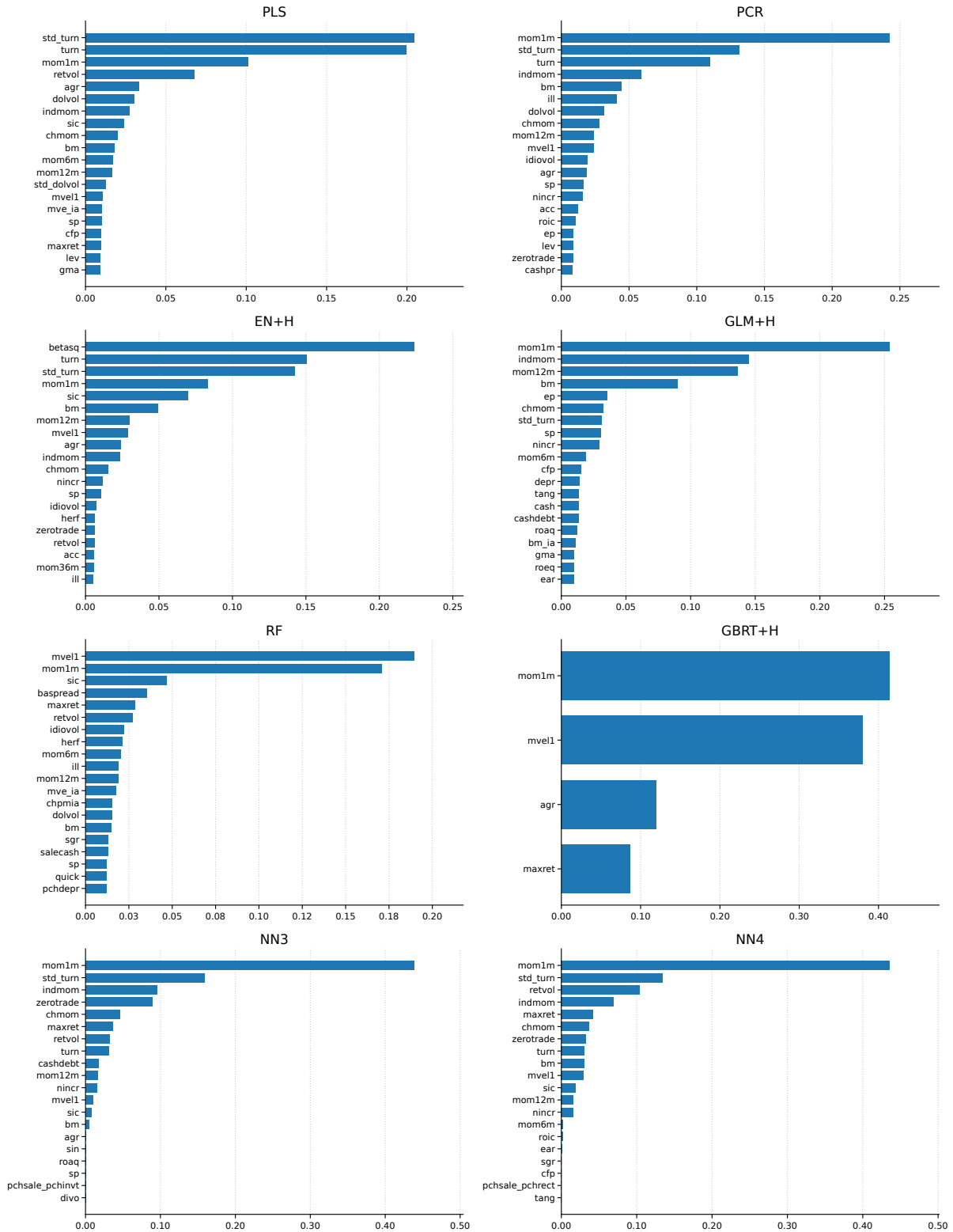
The momentum group dominates across nearly all specifications. Short-term momentum (`mom1m`) consistently ranks as the single most important predictor in almost every model, confirming the strong persistence of short-horizon return dynamics. Other momentum-related variables, such as industry momentum (`indmom`), twelve-month momentum (`mom12m`), and changes in six-month momentum (`chmom`), also play an important role, especially in the GLM and neural network models. This pattern underscores the central role of past returns, both at the firm and industry level, in explaining cross-sectional differences in expected returns.

The second major category consists of liquidity and trading activity measures, notably share turnover (`turn`) and the volatility of turnover (`std_turn`). These variables consistently rank among the most informative predictors in PLS, PCR, EN, and the neural networks, indicating that periods of intense or unstable trading activity convey valuable signals about future returns. In particular, PLS and PCR place strong emphasis on these liquidity proxies.

A third category corresponds to risk and volatility indicators, this includes return volatility (`retvol`) and squared beta (`betasq`). These variables capture exposure to systematic and idiosyncratic risk and appear among the most relevant predictors in several models. While `betasq` is particularly influential in the elastic net, `retvol` also plays an important role in PLS and in deeper neural networks (e.g., NN4), suggesting that measures of volatility contribute meaningfully to return predictability across both linear and nonlinear frameworks.

The firm size variable (`mvel1`) and industry dummies (`sic`) are especially important in tree-based methods such as random forest. These models also assign notable importance to microstructure-related variables, including the bid-ask spread (`baspread`) and the number of zero-trading days (`zerotrade`), which capture illiquidity and trading frictions that linear models may not easily accommodate.

Figure 8: Variable Importance by Model – Stock-Specific Characteristics



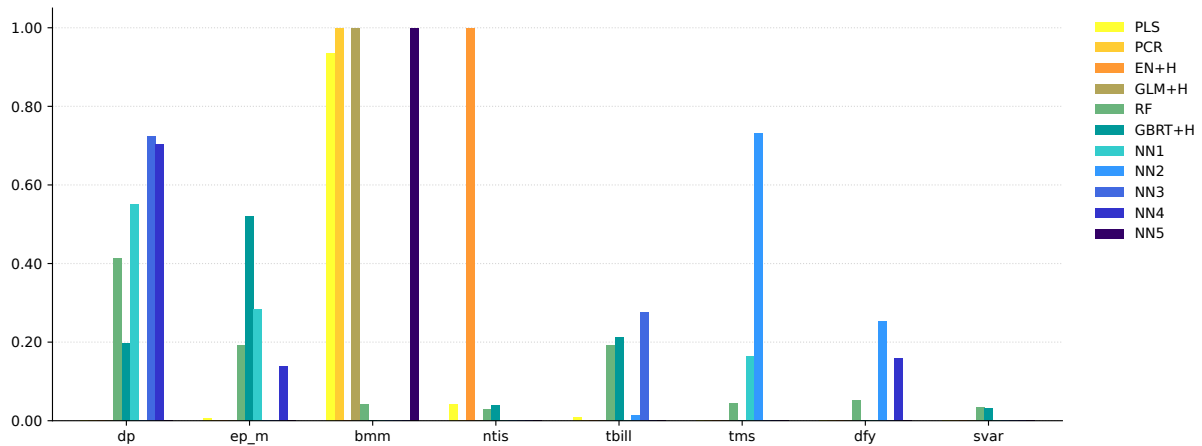
Note: Variable importance for the top 20 (when available) most influential stock characteristics in each model. Values represent averages across all training samples. Importances within each model are normalized to sum to one within the displayed set, ensuring comparability across models.

The gradient boosted regression tree (GBRT) model behaves differently. Only four stock-specific predictors—mom1m, mvel1, agr (asset growth), and maxret (maximum daily return)—emerge as directly relevant. This indicates that much of GBRT’s explanatory power arises from interactions or macroeconomic variables, rather than from individual characteristics alone.

Despite these methodological differences, the key takeaway is that momentum, liquidity, and volatility-related characteristics remain the most consistently important drivers of cross-sectional stock return predictability, in line with the findings of Gu et al. (2020), who similarly identify these categories as the dominant sources of predictive power across machine learning models.

Table 3: Variable Importance for Macroeconomic Predictors

	PLS	PCR	EN +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
dp	0.16	0.04	0.0	0.01	41.23	19.66	55.19	0.00	72.37	70.40	0.00
ep_m	0.73	0.19	0.0	0.06	19.18	51.92	28.30	0.00	0.00	13.77	0.21
bmm	93.51	99.76	0.0	99.84	4.26	0.00	0.00	0.00	0.00	0.00	99.79
ntis	4.22	0.01	100.0	0.03	2.92	4.00	0.00	0.00	0.00	0.00	0.00
tbill	1.00	0.00	0.0	0.00	19.23	21.29	0.00	1.47	27.63	0.00	0.00
tms	0.14	0.00	0.0	0.06	4.53	0.00	16.51	73.28	0.00	0.00	0.00
dfy	0.05	0.01	0.0	0.01	5.14	0.00	0.00	25.25	0.00	15.84	0.00
svar	0.18	0.00	0.0	0.00	3.51	3.14	0.00	0.00	0.00	0.00	0.00



Note: Variable importance for eight macroeconomic predictors across all models. Importance values represent the average contribution of each variable over all training samples, normalized within each model to sum to one. The lower panel provides a visual comparison of macroeconomic variable importances across models.

Table 3 reports the relative importance of the eight macroeconomic predictors across models. A few clear patterns emerge.

First, the book-to-market ratio (bmm) dominates the macroeconomic layer in the linear and dimension-reduction models. It accounts for nearly all the explanatory con-

tribution in PCR, PLS, and GLM, and captures almost the entire macroeconomic signal in NN5. This suggests that valuation-based measures remain the most reliable macro-level proxy for aggregate risk premia, regardless of whether the model is explicitly linear or implicitly regularized toward linearity.

Second, the net equity expansion (ntis), which captures corporate financing activity, plays a central role in the elastic net, where it receives the entire model weight (100%). This outcome reflects the tendency of strongly regularized linear models to concentrate predictive weight on a single variable, effectively shrinking all others toward zero.

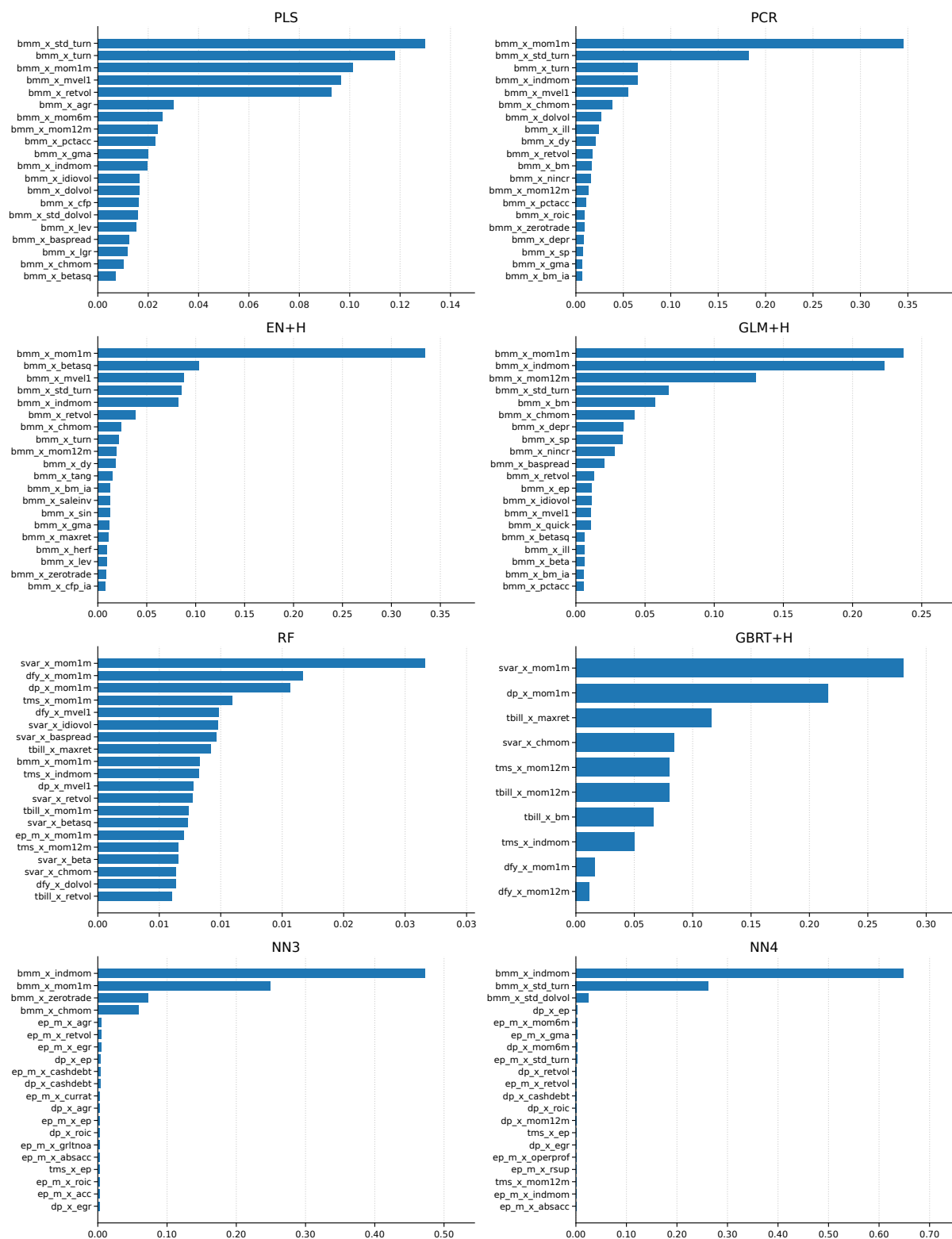
Tree-based models reveal a distinct but still concentrated structure. In random forest and GBRT, importance is shared mainly among a small set of macro variables, the dividend–price ratio (dp), earnings–price ratio (ep_m), and Treasury-bill rate (tbill), although their weights differ across the two models.

Among the neural networks, the relative weighting shifts substantially across architectures. Shallow networks (NN1–NN2) emphasize yield and valuation based factors, NN1 focusing on dp, ep_m, and tms, and NN2 on tms and dfy, whereas deeper networks (NN3–NN4) concentrate on dp, ep_m, and tbill. NN5, by contrast, collapses to a nearly linear pattern, assigning almost all explanatory weight to the book-to-market ratio (bmm), possibly reflecting over-regularization or convergence to a simpler functional form during training.

Overall, the results indicate that each model family captures a distinct but narrow subset of macroeconomic information. Valuation ratios (bmm, dp, ep_m) and interest rate variables (tbill, tms) consistently emerge as the dominant drivers of return predictability, though their relative influence varies across modeling approaches.

Figure 9 reports the variable importance for interaction terms across models.

Figure 9: Variable Importance by Model – Interactions



Note: Variable importance for the top 20 (when available) most influential interactions in each model. Values represent averages across all training samples. Importances within each model are normalized to sum to one within the displayed set, ensuring comparability across models.

5.4. Machine Learning Portfolios

While the previous sections focused on the statistical performance of the models, it is equally important to evaluate whether predictive accuracy translates into economically meaningful results. To this end, I construct portfolio strategies based on model forecasts of expected excess returns. At the end of each month, all stocks are sorted into ten deciles according to their predicted next-month returns, and value-weighted or equally-weighted portfolios are formed. The high-minus-low (H-L) portfolio — which takes a long position in the top decile (Decile 10) and a short position in the bottom decile (Decile 1) — measures the economic value of model forecasts in terms of realized returns.

This section presents the out-of-sample performance of these machine learning portfolios, examining their average returns, volatility, Sharpe ratios, and drawdowns. The analysis helps assess whether models that achieve higher predictive R^2_{oos} also generate superior investment performance.

Table 4 summarizes the out-of-sample performance of the value-weighted portfolios constructed from machine learning forecasts. For each method, the table reports average realized returns, volatility, and Sharpe ratios across return-sorted deciles, together with the high-minus-low (H-L) spread that captures the return difference between the long position in the top decile and the short position in the bottom decile.

Contrary to expectations based on predictive accuracy, portfolio performance does not align closely with the out-of-sample R^2 results. According to Gu et al. (2020), models achieving higher predictive R^2_{oos} are generally expected to yield superior investment performance. However, the evidence in Table 4 suggests otherwise. The generalized linear model (GLM+H), which recorded one of the lowest R^2_{oos} values—second only to OLS—produces the highest Sharpe ratio across all models. Similarly, the OLS+H specification, despite being the weakest performer in terms of predictive accuracy, attains the second-highest Sharpe ratio. By contrast, the PLS model shows internal consistency: it ranks among the top performers both in predictive R^2_{oos} and in portfolio Sharpe ratio, standing third overall. Meanwhile, the deep neural network (NN5), which achieved one of the best R^2_{oos} scores, ranks only fourth in terms of Sharpe ratio—comparable to the elastic net (EN+H). GBRT, on the other hand, is fairly consistent with the results; it is the worst portfolio in terms of SR, together with OLS-3+H.

In the context of value-weighted portfolios, statistical prediction accuracy does not necessarily translate into superior portfolio performance. This can be explained by the fact that the out-of-sample R^2 measures the absolute accuracy of predictions at the individual-stock level, whereas portfolio returns do not depend on how close each forecast is to the realized value, but rather on how well the model ranks stocks—that is, whether it places at the top those that subsequently deliver higher returns.

Table 4: Performance of Value Weighted Machine Learning Portfolios

	OLS+H				OLS-3+H				PLS			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	-0.45	0.28	5.93	0.16	-0.41	0.64	7.11	0.31	-1.04	0.36	6.38	0.2
2	0.83	0.54	5.29	0.35	-0.07	0.6	5.59	0.37	-0.43	0.58	5.73	0.35
3	1.63	0.64	4.85	0.46	0.17	0.69	4.81	0.5	-0.09	0.52	4.97	0.36
4	2.3	0.6	5.14	0.4	0.37	0.71	4.52	0.54	0.19	0.63	4.9	0.44
5	2.93	0.81	5.16	0.54	0.54	0.68	5.09	0.46	0.43	0.65	4.8	0.47
6	3.54	0.85	5.26	0.56	0.69	0.92	5.26	0.6	0.66	0.78	4.85	0.56
7	4.16	1.1	5.81	0.65	0.84	0.86	4.98	0.6	0.89	0.9	4.69	0.67
8	4.86	1.1	6.02	0.63	1.01	0.88	5.17	0.59	1.13	0.92	5.15	0.62
9	5.7	1.17	6.92	0.59	1.24	0.7	5.25	0.46	1.44	0.79	5.21	0.53
High(H)	7.04	1.15	7.54	0.53	1.55	0.93	6.25	0.51	1.94	1.01	5.65	0.62
H-L	7.5	0.87	6.99	0.43	1.97	0.29	6.68	0.15	2.98	0.65	5.55	0.41

	PCR				EN+H				GLM+H			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	-0.77	0.38	6.46	0.2	-1.51	0.22	7.41	0.1	0.14	-0.1	7.72	-0.05
2	-0.24	0.48	5.46	0.31	-1.06	0.56	5.85	0.33	0.57	0.38	6.59	0.2
3	0.08	0.62	4.93	0.44	-0.78	0.51	5.42	0.33	0.82	0.51	5.62	0.31
4	0.33	0.63	4.81	0.45	-0.56	0.67	4.96	0.47	1.0	0.72	5.06	0.49
5	0.55	0.76	4.63	0.57	-0.37	0.73	4.94	0.51	1.15	0.69	4.96	0.48
6	0.74	0.72	4.91	0.51	-0.18	0.77	4.63	0.58	1.29	0.79	4.57	0.6
7	0.94	0.73	5.14	0.49	0.0	0.78	4.62	0.58	1.43	0.97	4.82	0.7
8	1.16	0.95	5.02	0.65	0.19	0.87	4.98	0.61	1.57	1.01	4.66	0.75
9	1.43	0.79	5.57	0.49	0.43	1.02	4.99	0.71	1.74	1.0	4.99	0.69
High(H)	1.87	0.89	5.8	0.53	0.81	0.92	5.35	0.6	2.03	1.02	4.88	0.73
H-L	2.64	0.51	5.64	0.32	2.32	0.7	6.21	0.39	1.88	1.12	6.26	0.62

	RF				GBRT+H				NN1			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	0.36	0.32	7.48	0.15	0.37	0.64	4.44	0.5	-0.28	0.24	9.17	0.09
2	0.43	0.45	5.89	0.26	0.37	0.66	4.62	0.5	0.19	0.74	7.91	0.32
3	0.47	0.59	5.25	0.39	0.38	0.67	4.9	0.47	0.39	0.47	6.46	0.25
4	0.5	0.74	5.01	0.51	0.39	0.78	5.75	0.47	0.52	0.72	5.91	0.42
5	0.53	0.74	4.82	0.53	0.39	0.77	5.8	0.46	0.61	0.75	5.64	0.46
6	0.56	0.8	4.83	0.58	0.4	0.7	5.63	0.43	0.7	0.9	5.21	0.6
7	0.6	0.89	4.92	0.63	0.41	0.53	6.14	0.3	0.8	0.75	4.91	0.53
8	0.65	0.71	5.28	0.47	0.44	0.83	6.58	0.44	0.92	0.79	5.11	0.54
9	0.71	0.86	6.16	0.49	0.47	0.67	6.81	0.34	1.13	0.92	5.58	0.57
High(H)	0.83	0.82	7.49	0.38	0.54	0.85	7.32	0.4	1.61	1.03	6.59	0.54
H-L	0.47	0.5	6.75	0.26	0.17	0.21	4.85	0.15	1.89	0.78	7.73	0.35

	NN2				NN3				NN4			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	-0.49	0.63	10.18	0.21	-0.3	0.21	10.21	0.07	-0.35	0.16	9.73	0.06
2	0.07	0.59	8.81	0.23	0.21	0.48	7.98	0.21	0.19	0.2	7.75	0.09
3	0.33	0.76	7.36	0.36	0.43	0.64	6.93	0.32	0.42	0.47	6.59	0.25
4	0.49	0.74	5.68	0.45	0.56	0.52	5.92	0.3	0.55	0.66	5.64	0.41
5	0.61	0.58	5.48	0.36	0.66	0.73	5.17	0.49	0.64	0.6	4.97	0.42
6	0.71	0.61	5.04	0.42	0.75	0.79	4.83	0.57	0.71	0.75	4.94	0.52
7	0.81	0.67	4.79	0.48	0.84	0.75	4.58	0.57	0.79	0.69	4.84	0.5
8	0.92	0.82	4.95	0.57	0.94	0.67	4.83	0.48	0.88	0.83	5.01	0.58
9	1.07	0.73	5.32	0.47	1.09	0.71	4.81	0.51	1.01	0.81	5.15	0.54
High(H)	1.51	0.97	5.97	0.56	1.48	1.12	5.23	0.74	1.34	0.97	6.13	0.55
H-L	2.0	0.34	7.94	0.15	1.78	0.91	8.32	0.38	1.7	0.81	8.02	0.35

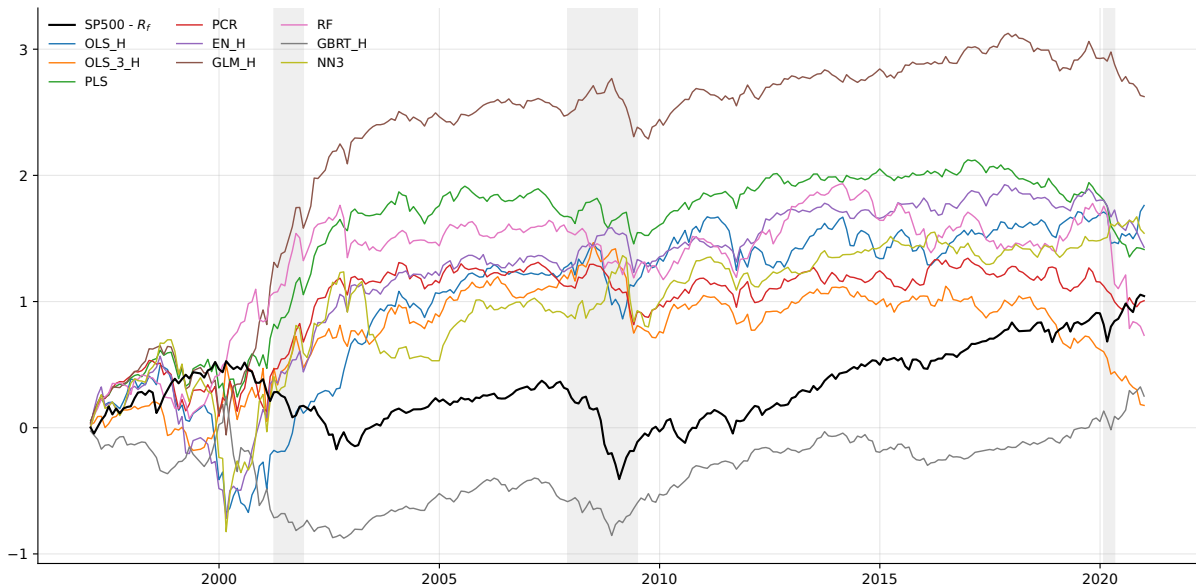
	NN5			
	Pred	Avg	Std	SR
Low(L)	0.05	0.37	9.22	0.14
2	0.41	0.41	7.48	0.19
3	0.55	0.34	6.12	0.19
4	0.63	0.44	5.28	0.29
5	0.67	0.8	4.74	0.59
6	0.71	0.85	4.76	0.62
7	0.74	0.65	4.72	0.48
8	0.78	0.69	4.97	0.48
9	0.85	0.75	5.38	0.48
High(H)	1.11	1.12	7.04	0.55
H-L	1.06	0.75	6.67	0.39

Note: I report prediction-sorted portfolio performance over the out-of-sample period. Stocks are sorted into deciles by next-month predicted returns. Columns “Pred”, “Avg”, “Std”, and “SR” show predicted returns, realized average monthly returns, standard deviations, and Sharpe ratios. Portfolios are value weighted.

In financial terms, it is also informative to evaluate portfolio performance through total compounded and annualized returns rather than monthly averages alone. Accordingly, Table 5 reports portfolios sorted from best to worst according to their compounded return, together with the cumulative log returns of the corresponding value-weighted strategies. This provides a long-horizon perspective on the economic gains implied by each model's forecasts.

Table 5: Compounded and Annualized Performance of Value-Weighted Portfolios

Model	Compounded Return (%)	Annualized Return (%)
GLM+H	1,279.73	11.60
OLS+H	482.16	7.64
NN3	367.16	6.66
NN5	346.52	6.46
EN+H	318.15	6.16
PLS	310.60	6.08
NN1	275.93	5.69
NN4	268.67	5.61
SP500- R_f	183.87	4.46
PCR	173.58	4.30
RF	108.07	3.11
GBRT+H	28.46	1.05
OLS-3+H	19.35	0.74
NN2	3.99	0.16



Note: The upper panel reports compounded and annualized portfolio returns for each model over the out-of-sample period. Portfolios are sorted in descending order according to compounded returns. Compounded returns represent total growth over the evaluation period, while annualized returns correspond to the equivalent average yearly rate. The lower panel displays cumulative log returns for the corresponding strategies. For clarity, only the best-performing neural network (NN) in terms of compounded return is shown. Shaded gray areas indicate NBER recession periods.

Table 5 shows that the GLM+H portfolio delivers by far the strongest long-run performance, achieving compounded and annualized returns more than twice those of the

OLS benchmark and roughly three times higher than the best-performing neural network. Several other models also record higher compounded returns than the market benchmark ($SP500-R_f$).

Table 6 reports performance for equally weighted prediction–sorted portfolios. Two patterns stand out.

First, moving from value to equal weights generally raises the economic significance of most model forecasts. Both average returns and Sharpe ratios increase for nearly all models except GBRT, which continues to underperform markedly. This highlights the limited predictive and economic value of the gradient boosting model in this setting, in contrast with the findings of Gu et al. (2020). NN5 achieves the highest Sharpe ratio, confirming the strong performance of deeper neural architectures when portfolios are equally weighted. All neural networks except NN1 deliver solid results, followed closely by PLS and PCR, which continue to perform competitively despite their simplicity. Interestingly, OLS maintains a relatively strong Sharpe ratio even under equal weighting, confirming that simple linear models can still deliver competitive portfolio performance.

Second, the link between statistical fit and portfolio payoffs is stronger here than under value weighting, but still imperfect. Models with better R^2_{os} (e.g., NN4–NN5, PLS/PCR) now tend to rank near the top by Sharpe ratio while OLS+H again delivers a surprisingly competitive Sharpe despite its negative R^2_{os} , underscoring that cross-sectional ranking power can matter more for portfolio performance than absolute forecast accuracy.

Table 7 reports the compounded and annualized performance of equally weighted portfolios. Compared to the value-weighted setting, both compounded and annualized returns are substantially higher across all models. Among all specifications, neural networks (particularly NN4) achieve the highest long-run growth, with compounded returns exceeding 40,000%. Surprisingly, the OLS+H model also delivers robust performance—outperforming NN1 and RF—and manages to retain strong profitability even under equal weighting, despite its relatively weak out-of-sample R^2 .

All models, with the sole exception of GBRT+H, outperform the market benchmark ($SP500-R_f$) in both compounded and annualized terms, further confirming the economic value of machine learning–based forecasts in this equal-weighted setting. Taken together, these results indicate that, when returns are equally weighted, most machine learning models are able to translate their cross-sectional predictive power into economically meaningful investment performance, although the magnitude of the gains varies considerably across architectures.

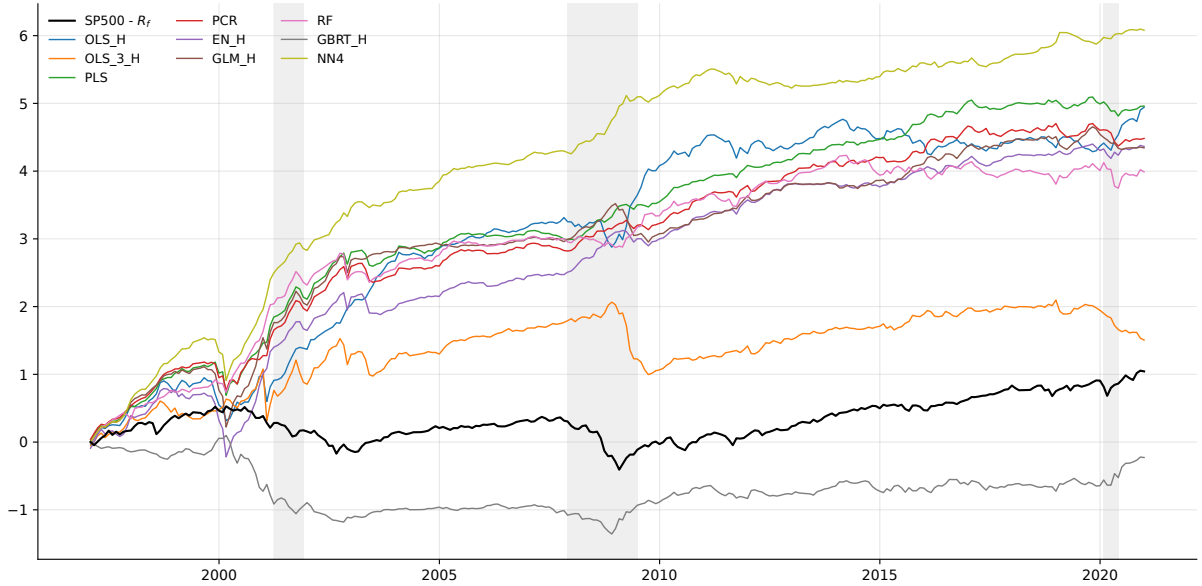
Table 6: Performance of Equally Weighted Machine Learning Portfolios

	OLS+H				OLS-3+H				PLS			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	-0.55	-0.16	6.36	-0.09	-0.42	0.59	10.0	0.21	-1.1	-0.15	8.25	-0.06
2	0.82	0.27	6.17	0.15	-0.07	0.63	7.84	0.28	-0.44	0.41	7.2	0.2
3	1.63	0.45	6.09	0.26	0.17	0.68	6.9	0.34	-0.09	0.65	6.64	0.34
4	2.31	0.76	6.04	0.43	0.37	0.78	6.51	0.42	0.19	0.76	6.12	0.43
5	2.94	0.83	6.16	0.47	0.54	0.74	6.19	0.41	0.43	0.81	5.82	0.48
6	3.55	0.93	6.11	0.53	0.69	0.86	5.74	0.52	0.66	1.0	5.66	0.61
7	4.18	1.0	6.26	0.55	0.84	0.96	5.25	0.63	0.89	0.97	5.7	0.59
8	4.88	1.26	6.64	0.66	1.02	0.94	4.91	0.66	1.14	1.12	5.58	0.69
9	5.75	1.5	7.19	0.72	1.25	1.1	4.88	0.78	1.45	1.36	5.92	0.79
High(H)	7.33	1.86	8.07	0.8	1.66	1.41	5.77	0.85	2.04	1.77	6.43	0.95
H-L	7.88	2.01	7.07	0.99	2.08	0.82	7.08	0.4	3.15	1.91	5.24	1.26
	PCR				EN+H				GLM+H			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	-0.86	-0.11	8.49	-0.04	-1.59	-0.21	8.81	-0.08	0.08	-0.29	9.09	-0.11
2	-0.25	0.41	7.23	0.2	-1.07	0.43	7.32	0.2	0.56	0.37	7.88	0.16
3	0.08	0.71	6.57	0.37	-0.79	0.73	6.72	0.38	0.81	0.69	7.02	0.34
4	0.33	0.8	6.18	0.45	-0.56	0.79	6.24	0.44	1.0	0.85	6.41	0.46
5	0.55	0.88	5.71	0.53	-0.37	0.91	6.11	0.51	1.15	0.93	6.04	0.54
6	0.74	0.98	5.56	0.61	-0.18	1.01	5.86	0.6	1.29	1.07	5.64	0.66
7	0.94	1.02	5.62	0.63	0.0	1.06	5.53	0.67	1.43	1.15	5.5	0.72
8	1.16	1.12	5.61	0.69	0.2	1.12	5.63	0.69	1.57	1.12	5.35	0.72
9	1.44	1.26	5.98	0.73	0.44	1.31	5.69	0.8	1.74	1.35	5.56	0.84
High(H)	1.95	1.63	6.39	0.88	0.88	1.54	5.97	0.89	2.06	1.46	5.58	0.91
H-L	2.81	1.74	5.26	1.15	2.46	1.75	6.11	0.99	1.98	1.76	6.34	0.96
	RF				GBRT+H				NN1			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	0.36	0.15	8.11	0.06	0.37	0.83	5.5	0.52	-0.4	-0.12	8.9	-0.04
2	0.43	0.56	6.77	0.29	0.37	0.8	5.37	0.52	0.18	0.5	7.55	0.23
3	0.47	0.68	5.85	0.4	0.38	0.86	5.21	0.57	0.39	0.8	6.68	0.41
4	0.5	0.7	5.59	0.44	0.39	0.79	5.55	0.49	0.51	0.82	6.14	0.46
5	0.53	0.73	5.69	0.44	0.39	0.91	5.78	0.54	0.61	0.99	5.84	0.59
6	0.56	0.82	5.65	0.51	0.4	0.91	6.0	0.52	0.7	1.02	5.61	0.63
7	0.6	0.96	5.52	0.6	0.42	0.95	6.52	0.51	0.8	1.11	5.35	0.72
8	0.65	1.01	6.08	0.57	0.44	1.06	6.98	0.52	0.93	1.03	5.43	0.66
9	0.72	1.34	6.62	0.7	0.47	0.74	7.72	0.33	1.16	1.03	6.0	0.6
High(H)	0.91	1.74	8.34	0.72	0.64	0.86	8.59	0.35	1.89	1.51	7.3	0.72
H-L	0.55	1.6	5.84	0.95	0.27	0.03	4.66	0.02	2.29	1.62	6.09	0.92
	NN2				NN3				NN4			
	Pred	Avg	Std	SR	Pred	Avg	Std	SR	Pred	Avg	Std	SR
Low(L)	-0.63	-0.22	9.26	-0.08	-0.42	-0.1	9.36	-0.04	-0.52	-0.3	9.11	-0.11
2	0.05	0.58	7.7	0.26	0.19	0.55	7.61	0.25	0.17	0.42	7.67	0.19
3	0.32	0.85	6.62	0.45	0.43	0.74	6.58	0.39	0.41	0.64	6.52	0.34
4	0.48	0.91	5.86	0.54	0.56	0.83	5.92	0.49	0.55	0.82	5.84	0.49
5	0.61	0.91	5.63	0.56	0.66	0.83	5.6	0.51	0.64	0.95	5.62	0.59
6	0.71	0.91	5.47	0.57	0.75	0.85	5.36	0.55	0.71	0.98	5.37	0.63
7	0.81	0.89	5.43	0.57	0.84	1.01	5.22	0.67	0.79	0.97	5.23	0.64
8	0.92	0.92	5.37	0.6	0.94	0.94	5.14	0.63	0.88	1.07	5.45	0.68
9	1.09	1.03	5.4	0.66	1.1	1.09	5.35	0.7	1.02	1.11	5.66	0.68
High(H)	1.81	1.92	7.41	0.9	1.88	1.96	8.08	0.84	1.75	2.02	8.13	0.86
H-L	2.43	2.14	5.07	1.46	2.31	2.06	5.22	1.37	2.27	2.32	5.38	1.49
	NN5											
	Pred	Avg	Std	SR								
Low(L)	-0.06	-0.2	9.19	-0.08								
2	0.4	0.58	7.9	0.25								
3	0.55	0.67	6.48	0.36								
4	0.63	0.76	5.64	0.47								
5	0.67	0.84	5.33	0.54								
6	0.71	0.96	5.11	0.65								
7	0.74	1.0	5.23	0.66								
8	0.78	1.02	5.42	0.65								
9	0.86	1.14	6.18	0.64								
High(H)	1.4	1.93	8.31	0.8								
H-L	1.47	2.13	4.83	1.53								

Note: I report prediction-sorted portfolio performance over the out-of-sample period. Stocks are sorted into deciles by next-month predicted returns. Columns “Pred”, “Avg”, “Std”, and “SR” show predicted returns, realized average monthly returns, standard deviations, and Sharpe ratios. Portfolios are equally weighted.

Table 7: Compounded and Annualized Performance of Equally-Weighted Portfolios

Model	Compounded Return (%)	Annualized Return (%)
NN4	43,579.33	28.94
NN2	27,543.65	26.50
NN5	27,612.07	26.51
NN3	21,601.38	25.23
PLS	14,168.65	23.05
OLS+H	13,921.95	22.96
PCR	8,745.19	20.61
EN+H	7,752.06	20.02
GLM+H	7,593.76	19.91
NN1	5,416.62	18.26
RF	5,302.43	18.15
OLS-3+H	350.57	6.50
SP500- R_f	183.87	4.46
GBRT+H	-20.39	-0.95



Note: The upper panel reports compounded and annualized portfolio returns for each model over the out-of-sample period. Portfolios are sorted in descending order according to compounded returns. Compounded returns represent total growth over the evaluation period, while annualized returns correspond to the equivalent average yearly rate. The lower panel displays cumulative log returns for the corresponding strategies. For clarity, only the best-performing neural network (NN) in terms of compounded return is shown. Shaded gray areas indicate NBER recession periods.

While return-based metrics provide insight into the profitability of the strategies, they do not fully capture their risk or implementation costs. To complement the performance analysis, I report the maximum drawdowns and portfolio turnover rates for each model. These measures offer a broader perspective on portfolio behavior by quantifying downside risk and the trading intensity implied by machine learning forecasts.

According to Gu et al. (2020), I define the maximum drawdown of a strategy as

$$\text{MaxDD} = \max_{0 \leq t_1 \leq t_2 \leq T} (Y_{t_1} - Y_{t_2}), \quad (28)$$

where Y_t denotes the cumulative log return from date 0 through t .

Maximum drawdown quantifies the largest observed decline in cumulative log returns before a new peak is reached. It captures the most severe loss an investor would have experienced over the evaluation period. Drawdowns highlight episodes in which cumulative performance falls from a prior maximum, providing a more intuitive and economically meaningful measure of downside risk than standard deviation.

The average monthly turnover measures how frequently portfolio holdings are adjusted over time. It captures the extent of rebalancing activity, where higher values indicate more frequent trading and thus higher transaction costs. Following Gu et al. (2020), the strategy’s average monthly turnover is defined as

$$\text{Turnover} = \frac{1}{T} \sum_{t=1}^T \left(\sum_i \left| w_{i,t+1} - \frac{w_{i,t}(1 + r_{i,t+1})}{\sum_j w_{j,t}(1 + r_{j,t+1})} \right| \right), \quad (29)$$

where $w_{i,t}$ denotes the portfolio weight of stock i at time t , and $r_{i,t+1}$ is its return between t and $t + 1$.

Table 8: Drawdowns and Turnover of Machine Learning Portfolios

	OLS +H	OLS-3 +H	PLS	PCR	EN +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
Drawdowns and Turnover (Value Weighted)													
Max DD(%)	69.11	72.47	53.72	39.02	72.36	50.72	69.91	67.45	86.66	78.24	78.16	74.28	60.52
Max 1M Loss(%)	27.93	34.34	18.09	21.09	25.15	23.25	34.63	21.19	37.87	35.75	44.08	42.00	22.94
Turnover(%)	163.21	134.75	166.15	157.0	175.37	166.27	174.71	121.89	170.78	166.53	170.83	176.64	181.41
Drawdowns and Turnover (Equally Weighted)													
Max DD(%)	47.01	65.73	38.79	34.30	63.54	59.00	38.35	76.67	75.37	43.96	46.67	46.71	31.07
Max 1M Loss(%)	30.78	53.34	29.28	19.40	34.05	36.39	31.65	18.16	34.16	21.85	24.70	31.68	20.81
Turnover(%)	146.46	122.89	153.22	146.31	162.11	155.07	157.25	121.81	157.73	156.55	158.37	163.01	167.56

Table 8 reports maximum drawdowns, maximum one-month losses, and average monthly turnover for both value and equally weighted portfolios.

In the value-weighted setting, maximum drawdowns tend to be relatively high across all models. For neural networks, drawdowns generally decrease as model complexity increases, while dimension-reduction approaches and the GLM achieve the lowest drawdowns (around 40–55%). Turnover levels remain considerably large for most methods, reflecting the frequent portfolio rebalancing implied by monthly re-training. In the equally weighted setting, both drawdowns and turnover decline for nearly all models, indicating that diversification across stocks helps reduce portfolio volatility and trading intensity.

In both cases, drawdowns are noticeably higher than those reported by Gu et al. (2020), suggesting that portfolio volatility is greater in this setting.

Although the results obtained in this study differ from those of Gu et al. (2020), it is evident that several of these models add value to portfolio decisions and overall per-

formance. However, estimating some of these models involves a considerable computational cost, especially for the more complex ones. This is mainly due to the repeated hyperparameter tuning required and the large scale of the dataset, which includes more than 800 predictors and nearly three million observations.

In this setting, even simple and computationally inexpensive models such as OLS+H deliver remarkably strong performance, achieving a compounded return of 13,921.95% and an annualized return of 22.96%, despite their limited statistical accuracy. Nevertheless, the best-performing neural network (NN4) still achieves over three times higher compounded returns (43,579.33%), suggesting that model flexibility may still play a role in enhancing long-run portfolio performance. This naturally raises the question: is it really necessary to estimate highly complex models such as neural networks to achieve such profitable results? To explore this issue, the next section introduces a simple benchmark strategy that relies solely on excess returns and involves minimal computational cost.

5.5. Z-Score Portfolio

The previous section’s results show that neural networks improve the effectiveness of stock selection in an equal-weighted portfolio setting. To assess whether such high portfolio returns can only be achieved through complex and computationally intensive models, this section introduces an approach based on a simple underlying idea.

The monthly excess return $r_{i,t}$ of a stock can be decomposed into a persistent component (related to fundamentals and risk) and a transitory component (shocks, liquidity frictions, over- or under-reaction, news). Each of these, in turn, consists of a common part (shared across the cross-section in a given month) and an idiosyncratic part (specific to each stock). The objective is to remove the month-specific common component to isolate the idiosyncratic part and then distinguish, even if imperfectly, stocks whose returns are more heavily driven by the transitory component than others.

To illustrate the underlying logic, consider a simple conceptual decomposition of the excess return:

$$r_{i,t} = P_{i,t} + T_{i,t}, \quad (30)$$

where $P_{i,t}$ represents the persistent component and $T_{i,t}$ the transitory one. This formulation is not intended as an estimable model but rather as a stylized representation of how different sources of return variation can be separated.

The persistent component can itself be decomposed into a stock-specific part, $SP_{i,t}$, which captures firm-level factors, and a common component, CP_t , that reflects market-wide or macroeconomic influences shared across all stocks:

$$P_{i,t} = SP_{i,t} + CP_t.$$

Analogously, the transitory component can be expressed as the sum of an idiosyncratic transitory element, $ST_{i,t}$, associated with firm-specific shocks, liquidity frictions, or short-term mispricing, and a common transitory component, CT_t , capturing temporary market-wide movements:

$$T_{i,t} = ST_{i,t} + CT_t.$$

Hence, the total excess return can be rewritten as

$$r_{i,t} = SP_{i,t} + CP_t + ST_{i,t} + CT_t. \quad (31)$$

Defining the cross-sectional z-score of excess returns as

$$\zeta_{i,t} = \frac{r_{i,t} - \mu_t}{\sigma_t}, \quad (32)$$

where μ_t and σ_t denote the cross-sectional mean and standard deviation of excess returns in month t , and if we assume that the common components CP_t and CT_t together correspond to the cross-sectional mean, i.e. $\mu_t = CP_t + CT_t$, then combining equation (32) with equation (31) yields

$$\zeta_{i,t} = \frac{SP_{i,t} + ST_{i,t}}{\sigma_t}. \quad (33)$$

The statistic $\zeta_{i,t}$ therefore measures how many cross-sectional standard deviations the excess return of stock i ($SP_{i,t} + ST_{i,t}$) deviates from the market mean ($\mu_t = CP_t + CT_t$) in month t . It provides a relative positioning measure, answering the question: how much better or worse did a stock perform compared to the average stock in the same month, in units of standard deviations? In other words, $\zeta_{i,t}$ expresses the relative performance of a stock, standardized by the cross-sectional dispersion of returns once the common monthly component has been removed.

Since σ_t is common to all stocks in the cross-section, the statistic $\zeta_{i,t}$ depends only on the stock-specific components $SP_{i,t}$ and $ST_{i,t}$. The focus is on extreme values of $\zeta_{i,t}$ corresponding to observations in the tails of the cross-sectional z-score distribution at each month t . The underlying intuition is that the persistent component of returns $SP_{i,t}$ typically evolves gradually over time, whereas the transitory component $ST_{i,t}$ may exhibit sharp deviations driven by temporary shocks, liquidity imbalances, or behavioral overreactions. Consequently, extreme $\zeta_{i,t}$ values are likely to identify stocks whose returns in month t are dominated by large transitory movements rather than by persistent, fundamental changes.

If these shocks are indeed transitory, their effects should gradually revert in subsequent periods: stocks that exhibit unusually high relative performance (high $\zeta_{i,t}$) are

expected to cool down, while those with unusually low relative performance (low $\zeta_{i,t}$) are likely to rebound. Building on this intuition, I construct a simple long–short portfolio that exploits these temporary extremes by taking long positions in strongly underperforming stocks and short positions in strongly outperforming ones.

Conceptually, the portfolio is a long–short “H–L” constructed on the tails of the cross-sectional z-score distribution rather than on deciles. At each month t , define the sets

$$H_t = \{i : \zeta_{i,t} \leq -k\}, \quad L_t = \{i : \zeta_{i,t} \geq k\}, \quad (34)$$

where $k > 0$ is a tail threshold.

The long–short return implemented at $t+1$ is

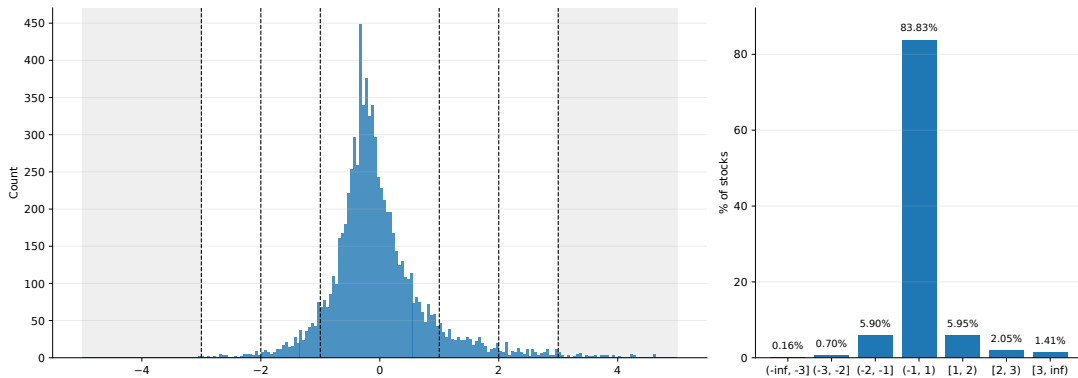
$$r_{t+1}^{H-L} = \frac{1}{|H_t|} \sum_{i \in H_t} r_{i,t+1} - \frac{1}{|L_t|} \sum_{i \in L_t} r_{i,t+1}.$$

with equal weights within each leg.

The threshold k can be viewed as a hyperparameter governing the aggressiveness of the strategy: smaller values include more stocks but capture less extreme deviations, whereas larger values focus on the most pronounced outliers at the cost of fewer positions. In principle, k could be selected adaptively using a validation set, but here I fix $k = 2$ for simplicity, so that only stocks with exceptionally low or high standardized returns enter the portfolio.

It’s important to note that in the decile-based portfolios, each leg includes roughly 10% of all stocks, so the H–L strategy involves about 20% of the cross section each month. In contrast, under the z-score rule, for a given cutoff k , only stocks satisfying the condition in equation (34) are included in the portfolio.

Figure 10: Cross-Sectional z-Score Distribution of Excess Returns (First Formation Month)

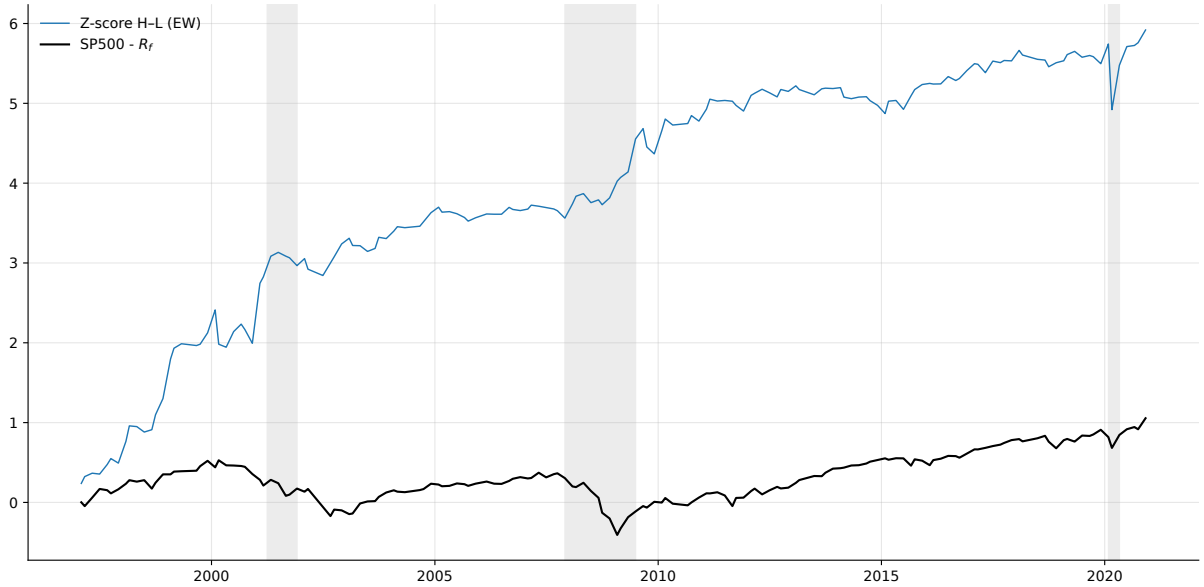


The left panel shows the empirical cross-sectional distribution of standardized excess returns ($\zeta_{i,t}$) in the first month used to form the z-score portfolios. The right panel reports the percentage of stocks falling within each z-score interval. The distribution changes across months as the cross-sectional dispersion of returns varies over time.

As illustrated in Figure 10, in the first month of portfolio formation only about 4.3% of stocks fall into the tails of the cross-sectional z-score distribution and are therefore included in the strategy ($0.16 + 0.70 + 2.05 + 1.41 \approx 4.3$). This fraction corresponds to the combined share of stocks exhibiting extreme standardized excess returns relative to the overall market distribution in that month. Over time, the tails of the cross-sectional z-score distribution become progressively thinner, with a smaller percentage of stocks exhibiting extreme standardized excess returns. This pattern suggests that market efficiency has increased in more recent years, as large relative mispricings or short-term dislocations have become less frequent.

Table 9: Performance of the Z-Score Portfolio (Equally Weighted)

Z-Score Portfolio (EW)	
Compounded Return (%)	37,100.25
Annualized Return (%)	28.19
Max DD (%)	56.08
Max 1M Loss (%)	56.08
Turnover (%)	196.09



Note: The upper panel reports compounded and annualized returns, maximum drawdown, maximum one-month loss, and average monthly turnover for the Z-score portfolio, constructed using equal weights. Compounded and annualized returns are computed over the out-of-sample period. The lower panel shows the cumulative log returns of the strategy compared with the market benchmark ($SP500 - R_f$). Shaded gray areas indicate NBER recession periods.

As reported in Table 9, the equally weighted Z-score portfolio ranks second overall, just below the best-performing neural network (NN4). It achieves remarkably strong compounded and annualized returns, with values comparable to those of the top machine learning models. The maximum drawdown of 56% coincides with the largest one-month loss, both occurring between January and February 2020, as shown in Ta-

ble 9. This episode aligns with the COVID-19 recession identified by the NBER and illustrates a key limitation of the strategy: it tends to slow down or underperform when extreme z-scores reflect genuine shifts in fundamentals rather than temporary shocks. In such phases, the tails no longer capture transitory mispricings but instead embody persistent information about changes in the economic environment. Nevertheless, the strategy typically recovers quickly once markets stabilize, as transient shocks again become the dominant drivers of short-term cross-sectional return variation.

The turnover of nearly 200% is consistent with the design of the strategy. Since portfolio positions are determined by the tails of the cross-sectional z-score distribution, the set of stocks included in each leg changes substantially from month to month. These shifts reflect the transient nature of the shocks captured by the strategy: extreme performers in one period rarely remain at the extremes in the next, leading to frequent rebalancing and a naturally high level of trading activity.

Considering its minimal computational cost and the absence of any model estimation, the performance of the Z-score portfolio is remarkably strong. Achieving results comparable to those of the best-performing neural network (NN4), it represents an appealing alternative when the primary objective is portfolio performance rather than understanding the variables driving the excess returns.

However, unlike machine learning methods, this approach does not learn or adapt to changing market conditions: it applies a fixed transformation of past excess returns that may exploit persistent patterns but cannot adjust when those patterns disappear or reverse. In a financial environment characterized by structural change and regime shifts, this distinction is essential, and will be further discussed in the next chapter.

6. Conclusions

This thesis set out to extend the analysis of Gu et al. (2020) by incorporating more recent data and excluding the earliest years of the original sample to assess the robustness and temporal stability of their results. The goal was to compare alternative modeling approaches in a high-dimensional asset pricing framework, identifying which methods deliver the best out-of-sample predictive accuracy and whether such improvements in statistical performance translate into tangible economic gains in portfolio construction.

In terms of predictive accuracy, neural networks emerged among the best performing models. Unlike the findings of Gu et al. (2020), however, deeper architectures (NN4 and NN5) outperformed their shallower counterparts, suggesting that model depth can be beneficial even in relatively noisy financial environments. Nevertheless, both networks failed to outperform the dimension-reduction methods, partial least squares (PLS) and principal components regression (PCR), in terms of overall R^2_{oos} , although they performed better within the Top 1000 and Bottom 1000 firm subsamples. In contrast to Gu et al. (2020), tree-based methods such as random forests and gradient-boosted regression trees performed poorly, providing limited predictive gains over simpler baselines.

Overall, the predictive performance observed in this study is weaker than that reported by Gu et al. (2020), despite the large number of observations used here. This discrepancy likely reflects increased market efficiency in more recent years: by removing the early part of the sample, the analysis excludes a period when return predictability was stronger and cross-sectional patterns were easier to exploit.

Portfolio-level results reveal a further distinction. For value-weighted portfolios, the link between statistical accuracy and economic performance is weak, as models with low predictive R^2_{oos} often achieve high Sharpe ratios. By contrast, in equally weighted portfolios, results align more closely with Gu et al. (2020): predictive accuracy and realized performance move more consistently together, although some discrepancies remain. For instance, the baseline OLS model, despite its low predictive power, still delivers competitive compounded returns.

Variable importance analyses show broad agreement across methods. Four main groups of characteristics consistently emerge as key drivers of expected returns: momentum, liquidity, risk and volatility, and firm size. These categories mirror those identified by Gu et al. (2020), reinforcing the conclusion that most machine learning models ultimately extract similar economic signals, even when their functional forms differ. At the macroeconomic level, valuation and yield related variables, particularly the book-to-market ratio, dividend-price ratio, and Treasury-bill rate, emerge as the

dominant sources of predictability across models. Linear and dimension-reduction approaches, as well as the deepest neural network (NN5), concentrate almost entirely on the book-to-market ratio, while tree-based methods and shallower networks assign substantial weight to a small subset of valuation and interest-rate variables. Overall, the evidence suggests that each model family extracts a distinct, but relatively narrow, macroeconomic signal rather than relying on a broad combination of predictors.

From an economic perspective, these findings highlight that the prediction of risk premia remains empirically challenging, particularly in more recent years. The relationship between statistical accuracy and portfolio performance proves to be weak in value-weighted portfolios but more consistent in equally weighted ones, indicating that predictive gains do not always translate directly into economic value. Overall, the evidence points to a structural decline in cross-sectional return predictability, consistent with the idea that improvements in market efficiency have progressively limited opportunities for return predictability.

My modest contribution is to demonstrate that a simple, model-free approach can achieve portfolio performance comparable to that of the best-performing neural network, while requiring a fraction of the computational cost. This result underscores that, in certain contexts, economic value can be extracted from the data without complex model estimation, provided that the underlying statistical structure of returns is exploited effectively.

The Z-score portfolio can achieve strong results, but its success may derive from exploiting persistent statistical patterns rather than from learning from data. By construction, it represents a fixed mapping of excess returns, as defined in equations 33–34, and therefore does not adapt to the prevailing economic regime. Financial markets, however, are dynamic systems in which stable patterns tend to disappear. The contribution of machine learning in this context is not to identify another temporary pattern, but to develop adaptive models that update as new information becomes available and remain effective as market conditions evolve. Neural networks, in particular, do not rely on a predetermined functional form but continuously learn from data, capturing changes in the underlying structure of returns that static models like the Z-score approach cannot accommodate.

Appendix

A. Details of the Characteristics

As reported in Table 10, the stock-specific characteristics used in this study are the same as those employed by Gu et al. (2020), with the exception of thirteen variables that were excluded from the dataset. The complete list of characteristics can be found in the Internet Appendix of their paper.

Table 10: Details of the Characteristics

No.	Acronym	Firm characteristic	Paper's author(s)	Year	Journal	Data Source	Frequency
1	absacc	Absolute accruals	Bandyopadhyay, Huang & Wirjanto	2010	WP	Compustat	Annual
2	acc	Working capital accruals	Sloan	1996	TAR	Compustat	Annual
3	aeaovol	Abnormal earnings announcement volume	Lerman, Livnat & Mendenhall	2007	WP	Compustat+CRSP	Quarterly
4	age	# years since first Compustat coverage	Jiang, Lee & Zhang	2005	RAS	Compustat	Annual
5	agr	Asset growth	Cooper, Gulen & Schill	2008	JF	Compustat	Annual
6	baspread	Bid-ask spread	Amihud & Mendelson	1989	JF	CRSP	Monthly
7	beta	Beta	Fama & MacBeth	1973	JPE	CRSP	Monthly
8	betasq	Beta squared	Fama & MacBeth	1973	JPE	CRSP	Monthly
9	bm	Book-to-market	Rosenberg, Reid & Lanstein	1985	JPM	Compustat+CRSP	Annual
10	bm_ia	Industry-adjusted book to market	Asness, Porter & Stevens	2000	WP	Compustat+CRSP	Annual
11	cash	Cash holdings	Palazzo	2012	JFE	Compustat	Quarterly
12	cashdebt	Cash flow to debt	Ou & Penman	1989	IAE	Compustat	Annual
13	cashpr	Cash productivity	Chandrashekar & Rao	2009	WP	Compustat	Annual
14	cfp	Cash flow to price ratio	Desai, Rajgopal & Venkatachalam	2004	TAR	Compustat	Annual
15	cfp_ia	Industry-adjusted cash flow to price ratio	Asness, Porter & Stevens	2000	WP	Compustat	Annual
16	chatoia	Industry-adjusted change in asset turnover	Soliman	2008	TAR	Compustat	Annual
17	chsho	Change in shares outstanding	Pontiff & Woodgate	2008	JF	Compustat	Annual
18	chempia	Industry-adjusted change in employees	Asness, Porter & Stevens	1994	WP	Compustat	Annual
19	chinv	Change in inventory	Thomas & Zhang	2002	RAS	Compustat	Annual
20	chmom	Change in 6-month momentum	Gettleman & Marks	2006	WP	CRSP	Monthly
21	chpmia	Industry-adjusted change in profit margin	Soliman	2008	TAR	Compustat	Annual
22	convind	Convertible debt indicator	Valta	2016	JFQA	Compustat	Annual
23	currat	Current ratio	Ou & Penman	1989	IAE	Compustat	Annual
24	depr	Depreciation / PP&E	Holthausen & Larcker	1992	IAE	Compustat	Annual
25	divi	Dividend initiation	Michaely, Thaler & Womack	1995	JF	Compustat	Annual
26	divo	Dividend omission	Michaely, Thaler & Womack	1995	JF	Compustat	Annual
27	dolvol	Dollar trading volume	Chordia, Subrahmanyam & Anshuman	2001	JFE	CRSP	Monthly
28	dy	Dividend to price	Litzenberger & Ramaswamy	1982	JF	Compustat	Annual
29	ear	Earnings announcement return	Kishore, Brandt, Santa-Clara & Venkatachalam	2008	WP	Compustat+CRSP	Quarterly

Table 11: Details of the Characteristics (Continued)

No.	Acronym	Firm characteristic	Paper's author(s)	Year	Journal	Data Source	Frequency
30	egr	Growth in common shareholder equity	Richardson, Sloan, Soliman & Tuna	2005	JAE	Compustat	Annual
31	ep	Earnings to price	Basu	1977	JF	Compustat	Annual
32	gma	Gross profitability	Novy-Marx	2013	JFE	Compustat	Annual
33	grCAPX	Growth in capital expenditures	Anderson & Garcia-Feijoo	2006	JF	Compustat	Annual
34	grltnoa	Growth in long term net operating assets	Fairfield, Whisenant & Yohn	2003	TAR	Compustat	Annual
35	herf	Industry sales concentration	Hou & Robinson	2006	JF	Compustat	Annual
36	hire	Employee growth rate	Bazdrech, Belo & Lin	2014	JPE	Compustat	Annual
37	idiovol	Idiosyncratic return volatility	Ali, Hwang & Trombley	2003	JFE	CRSP	Monthly
38	ill	Illiquidity	Amihud	2002	JFM	CRSP	Monthly
39	indmom	Industry momentum	Moskowitz & Grinblatt	1999	JF	CRSP	Monthly
40	invest	Capital expenditures and inventory	Chen & Zhang	2010	JF	Compustat	Annual
41	lev	Leverage	Bhandari	1988	JF	Compustat	Annual
42	lgr	Growth in long-term debt	Richardson, Sloan, Soliman & Tuna	2005	JAE	Compustat	Annual
43	maxret	Maximum daily return	Bali, Cakici & Whitelaw	2011	JFE	CRSP	Monthly
44	mom12m	12-month momentum	Jegadeesh	1990	JF	CRSP	Monthly
45	mom1m	1-month momentum	Jegadeesh & Titman	1993	JF	CRSP	Monthly
46	mom36m	36-month momentum	Jegadeesh & Titman	1993	JF	CRSP	Monthly
47	mom6m	6-month momentum	Jegadeesh & Titman	1993	JF	CRSP	Monthly
48	mvel1	Size	Banz	1981	JFE	CRSP	Monthly
49	mve_ia	Industry-adjusted size	Asness, Porter & Stevens	2000	WP	Compustat	Annual
50	nincr	Number of earnings increases	Barth, Elliott & Finn	1999	JAR	Compustat	Quarterly
51	operprof	Operating profitability	Fama & French	2015	JFE	Compustat	Annual
52	pchcapx_ia	Industry adjusted % change in capital expenditures	Abarbanell & Bushee	1998	TAR	Compustat	Annual
53	pchcurrat	% change in current ratio	Ou & Penman	1989	JAE	Compustat	Annual
54	pchdepr	% change in depreciation	Holthausen & Larcker	1992	JAE	Compustat	Annual
55	pchgm_pchsale	% change in gross margin – % change in sales	Abarbanell & Bushee	1998	TAR	Compustat	Annual
56	pchquick	% change in quick ratio	Ou & Penman	1989	JAE	Compustat	Annual
57	pchsale_pchinv	% change in sales – % change in inventory	Abarbanell & Bushee	1998	TAR	Compustat	Annual
58	pchsale_pchrect	% change in sales – % change in AR	Abarbanell & Bushee	1998	TAR	Compustat	Annual

Table 12: Details of the Characteristics (Continued)

No.	Acronym	Firm characteristic	Paper's author(s)	Year	Journal	Data Source	Frequency
59	pchsale_pchxsga	% change in sales – % change in SG&A	Abarbanell & Bushee	1998	TAR	Compustat	Annual
60	pchsaleinv	% change in sales-to-inventory	Ou & Penman	1989	JAE	Compustat	Annual
61	pctacc	Percent accruals	Hafzalla, Lundholm & Van Winkle	2011	TAR	Compustat	Annual
62	pricedelay	Price delay	Hou & Moskowitz	2005	RFS	CRSP	Monthly
63	quick	Quick ratio	Ou & Penman	1989	JAE	Compustat	Annual
64	retvol	Return volatility	Ang, Hodrick, Xing & Zhang	2006	JF	CRSP	Monthly
65	roaq	Return on assets	Balakrishnan, Bartov & Faurel	2010	JAE	Compustat	Quarterly
66	roeq	Return on equity	Hou, Xue & Zhang	2015	RFS	Compustat	Quarterly
67	roic	Return on invested capital	Brown & Rowe	2007	WP	Compustat	Annual
68	rsup	Revenue surprise	Kama	2009	JBFA	Compustat	Quarterly
69	salecash	Sales to cash	Ou & Penman	1989	JAE	Compustat	Annual
70	saleinv	Sales to inventory	Ou & Penman	1989	JAE	Compustat	Annual
71	salerec	Sales to receivables	Ou & Penman	1989	JAE	Compustat	Annual
72	securedind	Secured debt indicator	Valta	2016	JFQA	Compustat	Annual
73	sgr	Sales growth	Lakonishok, Shleifer & Vishny	1994	JF	Compustat	Annual
74	sin	Sin stocks	Hong & Kacperczyk	2009	JFE	Compustat	Annual
75	sp	Sales to price	Barbee, Mukherji & Raines	1996	FAJ	Compustat	Annual
76	std_dolvol	Volatility of liquidity (dollar trading volume)	Chordia, Subrahmanyam & Anshuman	2001	JFE	CRSP	Monthly
77	std_turn	Volatility of liquidity (share turnover)	Chordia, Subrahmanyam & Anshuman	2001	JFE	CRSP	Monthly
78	tang	Debt capacity / firm tangibility	Almeida & Campello	2007	RFS	Compustat	Annual
79	tb	Tax income to book income	Lev & Nissim	2004	TAR	Compustat	Annual
80	turn	Share turnover	Datar, Naik & Radcliffe	1998	JFM	CRSP	Monthly
81	zerotrade	Zero trading days	Liu	2006	JFE	CRSP	Monthly

B. Hyperparameters

As shown in Table 13, the hyperparameter configurations adopted in this thesis follow those used by Gu et al. (2020). The parameter grids and search ranges are therefore fully consistent with their original setup, ensuring comparability of results across models.

Table 13: Hyperparameters For All Methods

	OLS +H	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1 - NN5
Huber loss $\xi =$ 99.9% quantile	✓	✓	–	–	✓	✓	–	✓	–
Others			K	K	$\rho = 0.5$ $\lambda \in (10^{-4}, 10^{-1})$	#Knots = 3 $\lambda \in (10^{-4}, 10^{-1})$	Depth = 1 ~ 6 #Trees = 300 #Features in each split $\in \{3, 5, 10, 20, 30, 50, \dots\}$	Depth = 1 ~ 2 #Trees = 1 ~ 1000 Learning Rate LR $\in \{0.01, 0.1\}$ Batch Size = 10000 Epochs = 100 Patience = 5 Adam Para. = Default Ensemble = 10	L1 penalty $\lambda_1 \in (10^{-5}, 10^{-3})$ Learning Rate LR $\in \{0.001, 0.01\}$

The notation “Huber loss $\xi = 99.9\%$ quantile” indicates that the threshold parameter ξ is recalibrated once per re-estimation period by fitting an OLS model on the current training window, computing the absolute in sample residuals, and setting ξ equal to their 99.9th percentile. This ensures that the Huber loss behaves like OLS for approximately 99.9% of observations while limiting the influence of extreme outliers on parameter estimation.

C. Algorithms

C.1. Lasso, Ridge, Elastic Net, and Group Lasso

The accelerated proximal algorithm (APG) is used, see, e.g., Boyd and Parikh (2013) and Polson et al. (2015), which allows for efficient implementation of the elastic net, Lasso, Ridge regression, and Group Lasso for both l_2 and Huber losses. The regularized objective functions is

$$\mathcal{L}(\theta; \cdot) = \underbrace{\mathcal{L}(\theta)}_{\text{Loss Function}} + \underbrace{\phi(\theta; \cdot)}_{\text{Penalty}}, \quad (35)$$

where the dependence on the tuning parameters is omitted. Specifically,

$$\phi(\theta; \cdot) = \begin{cases} \frac{1}{2}\lambda \sum_{j=1}^P \theta_j^2, & \text{Ridge;} \\ \lambda \sum_{j=1}^P |\theta_j|, & \text{Lasso;} \\ \lambda(1 - \rho) \sum_{j=1}^P |\theta_j| + \frac{1}{2}\lambda\rho \sum_{j=1}^P \theta_j^2, & \text{Elastic Net;} \\ \lambda \sum_{j=1}^P \|\theta_j\|, & \text{Group Lasso.} \end{cases} \quad (36)$$

where in the Group Lasso case, $\theta = (\theta_1, \theta_2, \dots, \theta_P)$ is a $K \times P$ matrix.

Proximal algorithms are a class of algorithms for solving convex optimization problems, in which the base operation is evaluating the proximal operator of a function, i.e., solving a small convex optimization problem. In many cases, this smaller problem has a closed form solution. The proximal operator is defined as:

$$\text{prox}_{\gamma f}(\theta) = \arg \min_z \left\{ f(z) + \frac{1}{2\gamma} \|z - \theta\|^2 \right\}. \quad (37)$$

An important property of the proximal operator is that the minimizer of a convex function $f(\cdot)$ is a fixed point of $\text{prox}_f(\cdot)$, i.e., θ^* minimizes $f(\cdot)$ if and only if

$$\theta^* = \text{prox}_f(\theta^*). \quad (38)$$

The proximal gradient algorithm is designed to minimize an objective function of the form (35), where $\mathcal{L}(\theta)$ is a differentiable function of θ but $\phi(\theta; \cdot)$ is not. Using

properties of the proximal operator, one can show that θ^* minimizes (35) if and only if

$$\theta^* = \text{prox}_{\gamma\phi}(\theta^* - \gamma\nabla\mathcal{L}(\theta^*)). \quad (39)$$

This result motivates the first two iteration steps in Algorithm 1. The third step inside the while loop is a Nesterov momentum (Nesterov 1983) adjustment that accelerates convergence.

The optimization problem requires the proximal operators of $\phi(\theta; \cdot)$ in (36), which have closed forms:

$$\text{prox}_{\gamma\phi}(\theta) = \begin{cases} \frac{\theta}{1 + \lambda\gamma}, & \text{Ridge;} \\ \lambda S(\theta, \lambda\gamma), & \text{Lasso;} \\ \frac{1}{1 + \lambda\gamma\rho} S(\theta, (1 - \rho)\lambda\gamma), & \text{Elastic Net;} \\ (\tilde{S}(\theta_1, \lambda\gamma)^\top, \tilde{S}(\theta_2, \lambda\gamma)^\top, \dots, \tilde{S}(\theta_P, \lambda\gamma)^\top)^\top, & \text{Group Lasso.} \end{cases} \quad (40)$$

where $S(x, \mu)$ and $\tilde{S}(x, \mu)$ are vector-valued functions, whose i th components are defined by:

$$(S(x, \mu))_i = \begin{cases} x_i - \mu, & \text{if } x_i > 0 \text{ and } \mu < |x_i|; \\ x_i + \mu, & \text{if } x_i < 0 \text{ and } \mu < |x_i|; \\ 0, & \text{if } \mu \geq |x_i|; \end{cases} \quad (\tilde{S}(x, \mu))_i = \begin{cases} x_i - \mu \frac{x_i}{\|x_i\|}, & \text{if } \|x_i\| > \mu; \\ 0, & \text{if } \|x_i\| \leq \mu. \end{cases}$$

Note that $S(x, \mu)$ is the soft-thresholding operator, so the proximal algorithm is equivalent to the coordinate descent algorithm in the case of l_2 loss (see, e.g., Daubechies et al. 2004; Friedman et al. 2007). The proximal framework adopted here allows efficient implementation of Huber loss and convergence acceleration.

Algorithm 1 Accelerated Proximal Gradient Method

- 1: **Initialization:** $\theta_0 = 0, m = 0, \gamma$;
 - 2: **while** θ_m not converged **do**
 - 3: $\bar{\theta} \leftarrow \theta_m - \gamma\nabla\mathcal{L}(\theta)|_{\theta=\theta_m}$
 - 4: $\hat{\theta} \leftarrow \text{prox}_{\gamma\phi}(\bar{\theta})$
 - 5: $\theta_{m+1} \leftarrow \hat{\theta} + \frac{m}{m+3}(\hat{\theta} - \theta_m)$
 - 6: $m \leftarrow m + 1$
 - 7: **end while**
 - 8: **Result:** The final parameter estimate is θ_m .
-

Implementation notes. The Python implementation used in this thesis follows the Accelerated Proximal Gradient (APG) algorithm described in Algorithm 1. The core algorithmic structure — gradient step, proximal mapping, and Nesterov acceleration — is identical to the reference formulations. The following practical adjustments were introduced to improve stability and scalability without altering the theoretical framework: (i) A backtracking line search adaptively adjusts the step size γ to ensure monotone decrease of the composite objective. (ii) The convergence criterion is based on the ℓ_2 -norm of successive iterates with a tolerance level of 10^{-6} . (iii) Warm starts are employed across regularization levels and between consecutive training windows to accelerate convergence. Warm starts reuse the solution at the previous regularization level as initialization for the next one, which accelerates convergence along the regularization path without skipping any value of λ .

C.2. Tree, Gradient Boosted Tree, and Random Forest

Algorithm 2 is a greedy algorithm (see, e.g., Breiman et al. 1984) used to grow a complete binary regression tree. Next, Algorithm 3 yields the random forest (e.g., Hastie 2009). Finally, Algorithm 4 delivers the gradient boosted tree (Friedman, 2001), for which the version written by Bühlmann and Hothorn (2007) is followed.

Algorithm 2 Classification and Regression Tree

- 1: **Initialize the stump.** $C_1(0)$ denotes the range of all covariates, and $C_l(d)$ denotes the l -th node of depth d .
- 2: **for** d from 1 to L **do**
- 3: **for** i in $\{C_l(d-1), l = 1, \dots, 2^{d-1}\}$ **do**
- 4: i) For each feature $j = 1, 2, \dots, P$, and each threshold level α , define a split $s = (j, \alpha)$, which divides $C_l(d-1)$ into C_{left} and C_{right} :

$$C_{\text{left}}(s) = \{z_j \leq \alpha\} \cap C_l(d-1), \quad C_{\text{right}}(s) = \{z_j > \alpha\} \cap C_l(d-1),$$

where z_j denotes the j -th covariate.

- 5: ii) Define the impurity function:

$$\mathcal{L}(C, C_{\text{left}}, C_{\text{right}}) = \frac{|C_{\text{left}}|}{|C|} H(C_{\text{left}}) + \frac{|C_{\text{right}}|}{|C|} H(C_{\text{right}}),$$

where

$$H(C) = \frac{1}{|C|} \sum_{z_{i,t} \in C} (r_{i,t+1} - \theta)^2, \quad \theta = \frac{1}{|C|} \sum_{z_{i,t} \in C} r_{i,t+1},$$

and $|C|$ denotes the number of observations in set C .

- 6: iii) Select the optimal split:

$$s^* \leftarrow \arg \min_s \mathcal{L}(C(s), C_{\text{left}}(s), C_{\text{right}}(s)).$$

- 7: iv) Update the nodes:

$$C_{2l-1}(d) \leftarrow C_{\text{left}}(s^*), \quad C_{2l}(d) \leftarrow C_{\text{right}}(s^*).$$

- 8: **end for**
- 9: **end for**

- 10: **Result:** The output of a regression tree is given by:

$$g(z_{i,t}; \theta, L) = \sum_{k=1}^{2^L} \theta_k 1\{z_{i,t} \in C_k(L)\}, \quad \text{where} \quad \theta_k = \frac{1}{|C_k(L)|} \sum_{z_{i,t} \in C_k(L)} r_{i,t+1}.$$

- 11: For a single binary complete regression tree $\mathcal{T}$ of depth L , the variable importance (VIP) for covariate z_j is:

$$\text{VIP}(z_j, \mathcal{T}) = \sum_{d=1}^{L-1} \sum_{i=1}^{2^{d-1}} \Delta \text{im}(C_i(d-1), C_{2i-1}(d), C_{2i}(d)) 1\{z_j \in \mathcal{T}(i, d)\},$$

where $\mathcal{T}(i, d)$ represents the covariate on the i -th internal node of depth d , which splits $C_i(d-1)$ into two sub-regions $\{C_{2i-1}(d), C_{2i}(d)\}$, and $\Delta \text{im}(\cdot, \cdot, \cdot)$ is defined by:

$$\Delta \text{im}(C, C_{\text{left}}, C_{\text{right}}) = H(C) - \mathcal{L}(C, C_{\text{left}}, C_{\text{right}}).$$

Algorithm 3 Random Forest

- 1: **for** b from 1 to B **do**
- 2: Generate Bootstrap samples $\{(z_{i,t}, r_{i,t+1}), (i, t) \in \text{Bootstrap}(b)\}$ from the original dataset, for which a tree is grown using **Algorithm 2**. At each step of splitting, use only a random subsample, say $\sqrt{P}$, of all features. Write the resulting b th tree as:

$$\hat{g}_b(z_{i,t}; \hat{\theta}_b, L) = \sum_{k=1}^{2^L} \hat{\theta}_b^{(k)} 1\{z_{i,t} \in C_k(L)\}.$$

- 3: **end for**
- 4: **Result:** The final random forest output is given by the average of the outputs of all B trees:

$$\hat{g}(z_{i,t}; L, B) = \frac{1}{B} \sum_{b=1}^B \hat{g}_b(z_{i,t}; \hat{\theta}_b, L).$$

Algorithm 4 Gradient Boosted Tree

- 1: Initialize the predictor as $\hat{g}_0(\cdot) = 0$;
- 2: **for** b from 1 to B **do**
- 3: Compute for each $i = 1, 2, \dots, N$ and $t = 1, 2, \dots, T$, the negative gradient of the loss function $l(\cdot, \cdot)$:

$$\varepsilon_{i,t+1} \leftarrow - \left. \frac{\partial l(r_{i,t+1}, g)}{\partial g} \right|_{g=\hat{g}_{b-1}(z_{i,t})}.$$

- 4: Grow a (shallow) regression tree of depth L with dataset $\{(z_{i,t}, \varepsilon_{i,t+1}) : \forall i, \forall t\}$:

$$\hat{f}_b(\cdot) \leftarrow g(z_{i,t}; \theta, L).$$

- 5: Update the model by

$$\hat{g}_b(\cdot) \leftarrow \hat{g}_{b-1}(\cdot) + \nu \hat{f}_b(\cdot),$$

where $\nu \in (0, 1]$ is a tuning parameter that controls the step length.

- 6: **end for**
- 7: **Result:** The final model output is

$$\hat{g}_B(z_{i,t}; B, \nu, L) = \sum_{b=1}^B \nu \hat{f}_b(\cdot).$$

C.3. Neural Networks

It is common to fit neural networks using stochastic gradient descent (SGD), see, e.g., Goodfellow et al. (2016). The adaptive moment estimation algorithm (Adam), an efficient version of SGD introduced by Kingma (2014), is employed to optimize the model parameters. Adam computes adaptive learning rates for individual parameters using estimates of the first and second moments of the gradients. The loss function is denoted as $\mathcal{L}(\theta; \cdot)$ and can be expressed as

$$\mathcal{L}(\theta; \cdot) = \frac{1}{T} \sum_{t=1}^T \mathcal{L}_t(\theta; \cdot),$$

where $\mathcal{L}_t(\theta; \cdot)$ represents the penalized cross-sectional average prediction error at time t . At each training step, a batch of observations is randomly sampled from the training dataset. Algorithm 6 describes the early stopping procedure, which can be applied in combination with many optimization routines, including Adam. Algorithm 7 presents the Batch-Normalization transform (Ioffe and Szegedy (2015)), applied to each activation following the ReLU transformation. Any neuron that previously received a batch of x as input now receives $\text{BN}_{\gamma, \beta}(x)$ instead, where γ and β are additional parameters to be optimized.

Algorithm 5 Adam for Stochastic Gradient Descent (SGD)

- 1: Initialize the parameter vector θ_0 . Set $m_0 = 0$, $v_0 = 0$, $t = 0$.
 - 2: **while** θ_t not converged **do**
 - 3: $t \leftarrow t + 1$.
 - 4: $g_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta; \cdot)|_{\theta=\theta_{t-1}}$.
 - 5: $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$.
 - 6: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t$.
 - 7: $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$.
 - 8: $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$.
 - 9: $\theta_t \leftarrow \theta_{t-1} - \alpha \hat{m}_t \oslash (\sqrt{\hat{v}_t} + \epsilon)^{-1}$.
 - 10: **end while**
 - 11: **Result:** The final parameter estimate is θ_t .
-

The symbols $\odot$ and $\oslash$ denote element-wise multiplication and division, respectively.

Algorithm 6 Early Stopping

- 1: Initialize $j = 0$, $\epsilon = \infty$ and select the patience parameter p .
 - 2: **while** $j < p$ **do**
 - 3: Update θ using the training algorithm (e.g., the steps inside the while loop of Algorithm 5 for h steps).
 - 4: Calculate the prediction error from the validation sample, denoted as ϵ' .
 - 5: **if** $\epsilon' < \epsilon$ **then**
 - 6: $j \leftarrow 0$.
 - 7: $\epsilon \leftarrow \epsilon'$.
 - 8: $\theta' \leftarrow \theta$.
 - 9: **else**
 - 10: $j \leftarrow j + 1$.
 - 11: **end if**
 - 12: **end while**
 - 13: **Result:** The final parameter estimate is θ' .
-

Algorithm 7 Batch Normalization (for one Activation over one Batch)

- 1: **Input:** Values of x for each activation over a batch $\mathcal{B} = \{x_1, x_2, \dots, x_N\}$.
- 2:

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{N} \sum_{i=1}^N x_i$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta := \text{BN}_{\gamma, \beta}(x_i)$$

- 3: **Result:** $\{y_i = \text{BN}_{\gamma, \beta}(x_i) : i = 1, 2, \dots, N\}$.
-

References

- Boyd, S. and Parikh, N. (2013). Proximal algorithms. *Foundations and Trends in optimization*, 1(3):123–231.
- Breiman, Friedman, Leo, J. H., Olshen, R. A., and Stone, C. J. (1984). Classification and regression trees. *Wadsworth International Group*, 8:452–456.
- Breiman, L. (2001). Random forests. *Machine learning*, 45(1):5–32.
- Bühlmann, P. and Hothorn, T. (2007). Boosting algorithms: Regularization, prediction and model fitting. *Statistical Science*.
- Butaru, F., Chen, Q., Clark, B., Das, S., Lo, A. W., and Siddique, A. (2016). Risk and risk management in the credit card industry. *Journal of Banking & Finance*, 72:218–239.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314.
- Daubechies, I., Defrise, M., and De Mol, C. (2004). An iterative thresholding algorithm for linear inverse problems with a sparsity constraint. *Communications on Pure and Applied Mathematics: A Journal Issued by the Courant Institute of Mathematical Sciences*, 57(11):1413–1457.
- De Jong, S. (1993). Simpls: an alternative approach to partial least squares regression. *Chemometrics and intelligent laboratory systems*, 18(3):251–263.
- Diebold, F. X. and Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263.
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *International workshop on multiple classifier systems*, pages 1–15. Springer.
- Eldan, R. and Shamir, O. (2016). The power of depth for feedforward neural networks. *Conference on learning theory*, pages 907–940.
- Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The journal of Finance*, 25(2):383–417.
- Fama, E. F. and French, K. R. (2008). Dissecting anomalies. *The journal of finance*, 63(4):1653–1678.

- Freund, Y. (1995). Boosting a weak learning algorithm by majority. *Information and computation*, 121(2):256–285.
- Freyberger, J., Neuhierl, A., and Weber, M. (2020). Dissecting characteristics nonparametrically. *The Review of Financial Studies*, 33(5):2326–2377.
- Friedman, J., Hastie, T., Höfling, H., and Tibshirani, R. (2007). Pathwise coordinate optimization. *The Annals of Applied Statistics*, 1:302–332.
- Friedman, J., Hastie, T., and Tibshirani, R. (2000). Special invited paper. additive logistic regression: A statistical view of boosting. *Annals of statistics*, pages 337–374.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232.
- Giglio, S. and Xiu, D. (2021). Asset pricing with omitted factors. *Journal of Political Economy*, 129(7):1947–1990.
- Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). *Deep learning*. MIT press Cambridge.
- Gu, S., Kelly, B., and Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273.
- Hansen, L. K. and Salamon, P. (2002). Neural network ensembles. *IEEE transactions on pattern analysis and machine intelligence*, 12(10):993–1001.
- Hastie, T. (2009). The elements of statistical learning: data mining, inference, and prediction.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.
- Heaton, J. B., Polson, N. G., and Witte, J. H. (2016). Deep learning in finance. *arXiv preprint arXiv:1602.06561*.
- Hinton, G. E., Osindero, S., and Teh, Y.-W. (2006). A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554.
- Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366.
- Hutchinson, J. M., Lo, A. W., and Poggio, T. (1994). A nonparametric approach to pricing and hedging derivative securities via learning networks. *The journal of Finance*, 49(3):851–889.

- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr.
- Kelly, B. and Pruitt, S. (2013). Market expectations in the cross-section of present values. *The Journal of Finance*, 68(5):1721–1756.
- Kelly, B. and Pruitt, S. (2015). The three-pass regression filter: A new approach to forecasting using many predictors. *Journal of Econometrics*, 186(2):294–316.
- Khandani, A. E., Kim, A. J., and Lo, A. W. (2010). Consumer credit-risk models via machine-learning algorithms. *Journal of Banking & Finance*, 34(11):2767–2787.
- Kingma, D. P. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Koijen, R. S. and Van Nieuwerburgh, S. (2011). Predictability of returns and cash flows. *Annu. Rev. Financ. Econ.*, 3(1):467–491.
- Kozak, S., Nagel, S., and Santosh, S. (2020). Shrinking the cross-section. *Journal of Financial Economics*, 135(2):271–292.
- Lewellen, J. (2015). The cross section of expected stock returns. *Critical Finance Review*, 4:1–44.
- Masters, T. (1993). *Practical neural network recipes in C++*. Morgan Kaufmann.
- Moritz, B. and Zimmermann, T. (2016). Tree-based conditional portfolio sorts: The relation between past and future stock returns. *Available at SSRN 2740751*.
- Nesterov, Y. (1983). A method for solving the convex programming problem with convergence rate $O(1/k^2)$. In *Dokl akad nauk Sssr*, volume 269, page 543.
- Polson, N. G., Scott, J. G., and Willard, B. T. (2015). Proximal algorithms in statistics and machine learning. *Statistical Science*, pages 559–581.
- Rapach, D. and Zhou, G. (2013). Forecasting stock returns. In *Handbook of economic forecasting*, volume 2, pages 328–383. Elsevier.
- Rapach, D. E., Strauss, J. K., and Zhou, G. (2013). International stock return predictability: What is the role of the united states? *The Journal of Finance*, 68(4):1633–1662.
- Sadhwani, A., Giesecke, K., and Sirignano, J. (2021). Deep learning for mortgage risk. *Journal of Financial Econometrics*, 19(2):313–368.
- Schapire, R. E. (1990). The strength of weak learnability. *Machine learning*, 5(2):197–227.

- Welch, I. and Goyal, A. (2008). A comprehensive look at the empirical performance of equity premium prediction. *The Review of Financial Studies*, 21(4):1455–1508.
- Yao, J., Li, Y., and Tan, C. L. (2000). Option price forecasting using neural networks. *Omega*, 28(4):455–466.